

EEE 475/575 Medical Image Reconstruction and Processing

Homework 4

Özgür Ekin Sönmez

29 April 2026

GenAI Usage Disclosure

I used OpenAI Codex (GPT-5) as a coding and writing assistant for this homework. The tool was used to draft the MATLAB implementation, organize the solution into runnable cells, create helper functions for reconstruction and visualization, and prepare this LaTeX report template. I reviewed and ran the code myself and remain responsible for the correctness, interpretation, and final submitted material.

Part I – Multi-Coil Reconstruction

1.1 Display MRI Images

The magnitude images show the spatially varying signal received by each coil. The phase images differ substantially across coils because each channel has its own complex sensitivity profile. The k-space spectra have most of their energy concentrated near the center, as expected for MR images. Comparing `map1` and `map2`, both maps have smooth coil sensitivity profiles, but `map2` contains small magnitude and phase mismatches. The measured mean absolute difference was 0.359498. Because OLC and SENSE use the sensitivity maps directly, these spatially varying mismatches can cause structured reconstruction errors, especially in SENSE.

Part 1.1: coil image magnitude

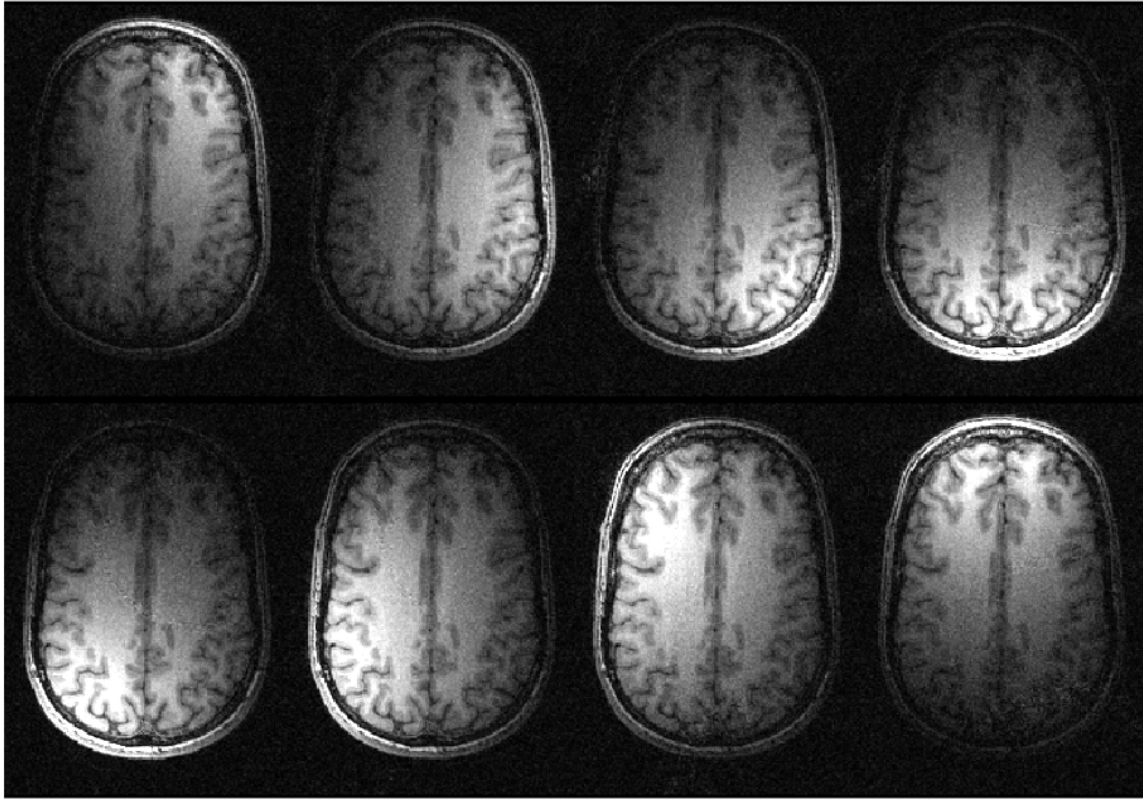


Figure 1: Magnitude of the fully sampled coil images.

Part 1.1: coil image phase

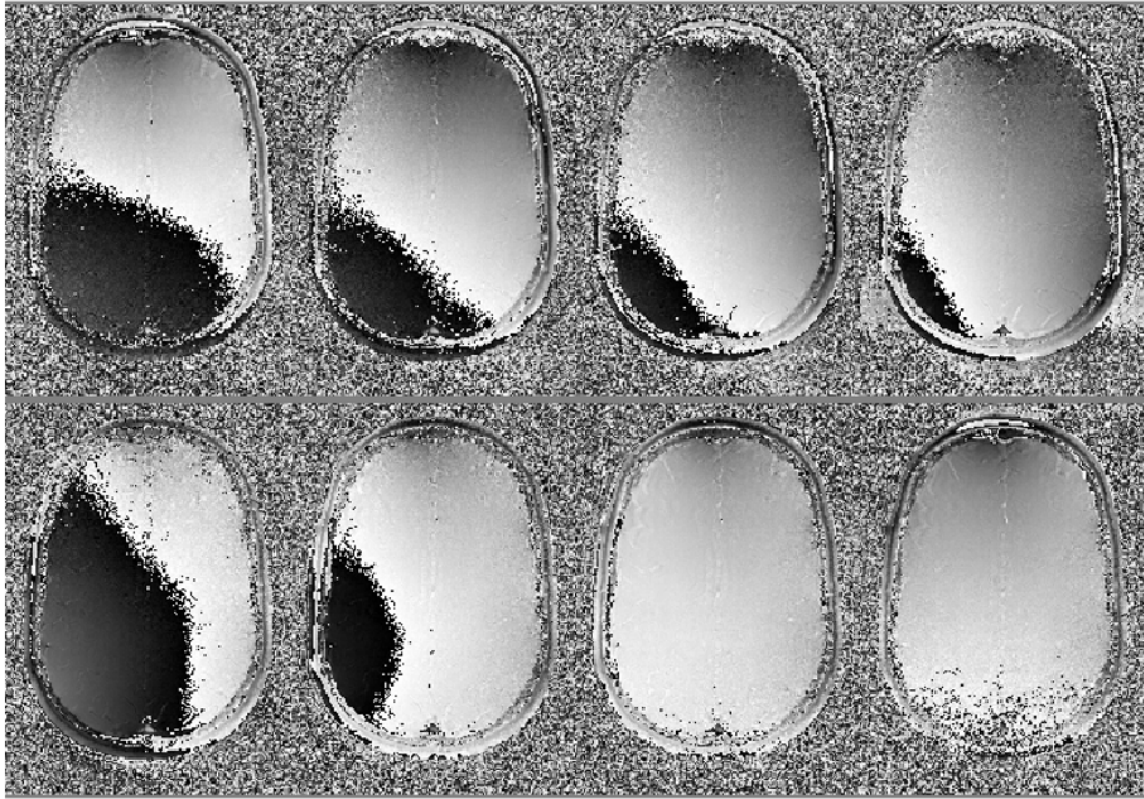


Figure 2: Phase of the fully sampled coil images.

Part 1.1: coil k-space spectra

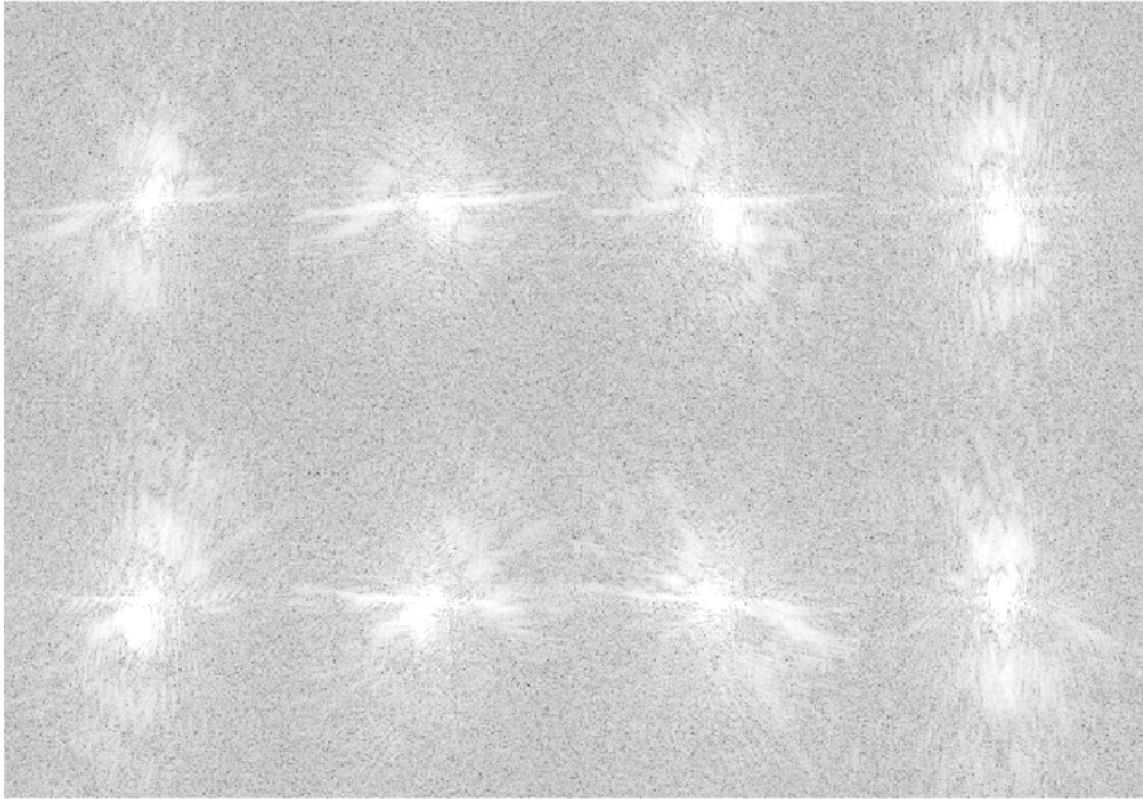


Figure 3: Log-scaled k-space spectra of the coil images.

Part 1.1: map1 sensitivity magnitude

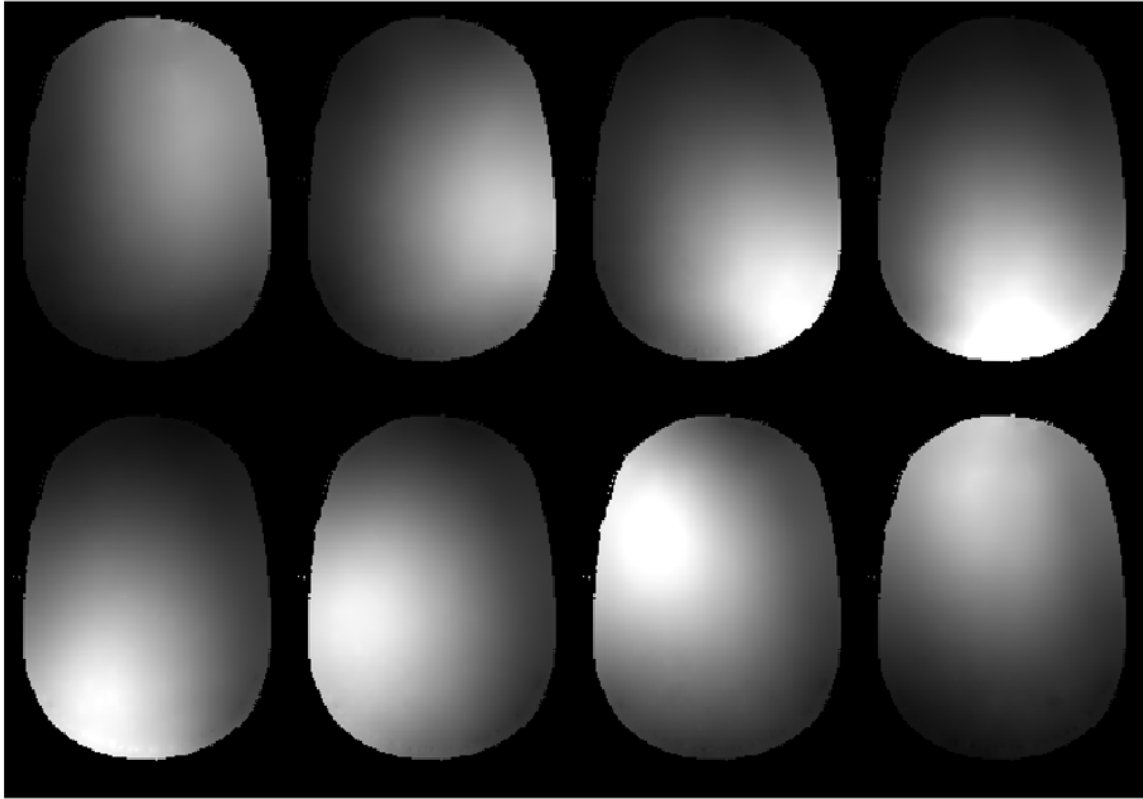


Figure 4: Magnitude of coil sensitivity map1.

Part 1.1: map1 sensitivity phase

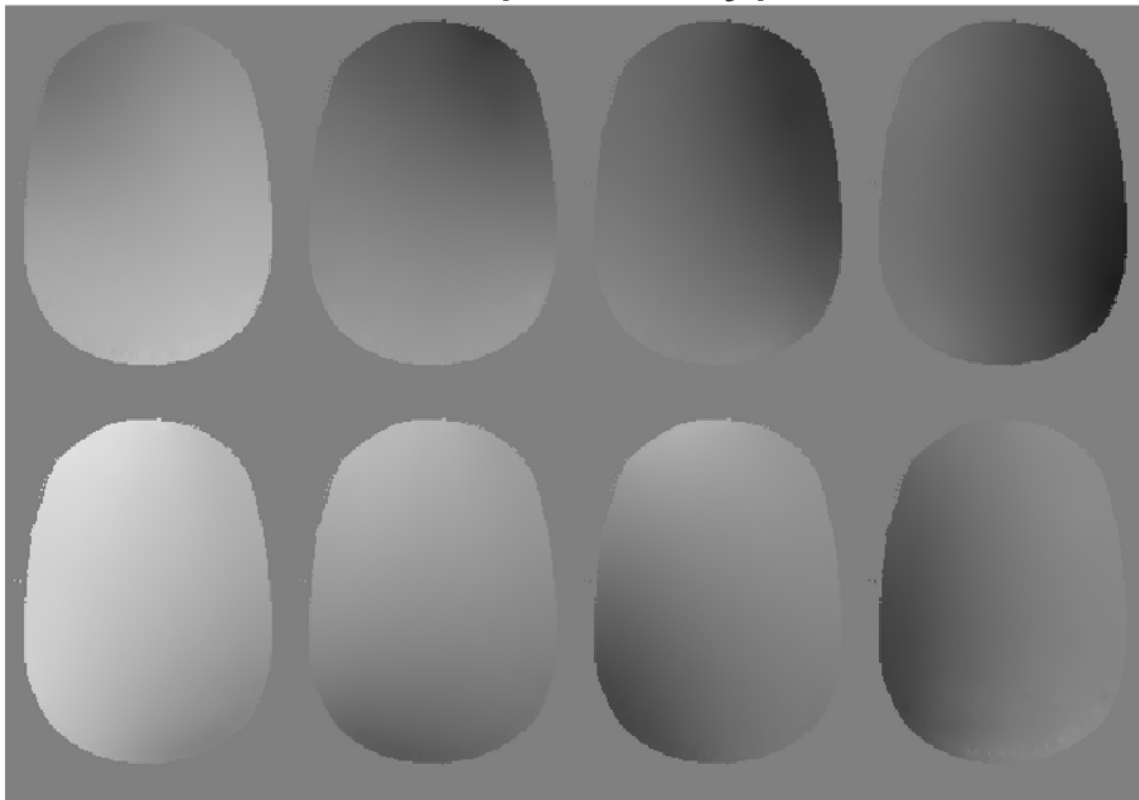


Figure 5: Phase of coil sensitivity map1.

Part 1.1: map2 sensitivity magnitude

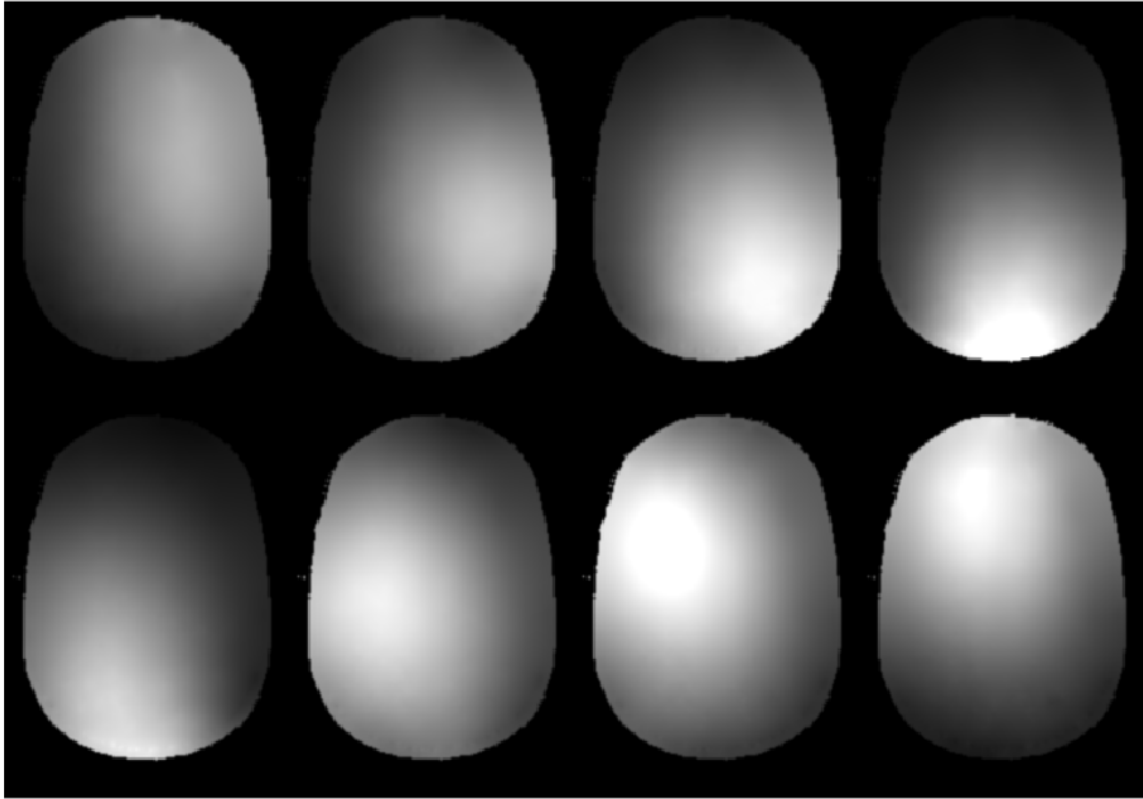


Figure 6: Magnitude of coil sensitivity map2.

Part 1.1: map2 sensitivity phase

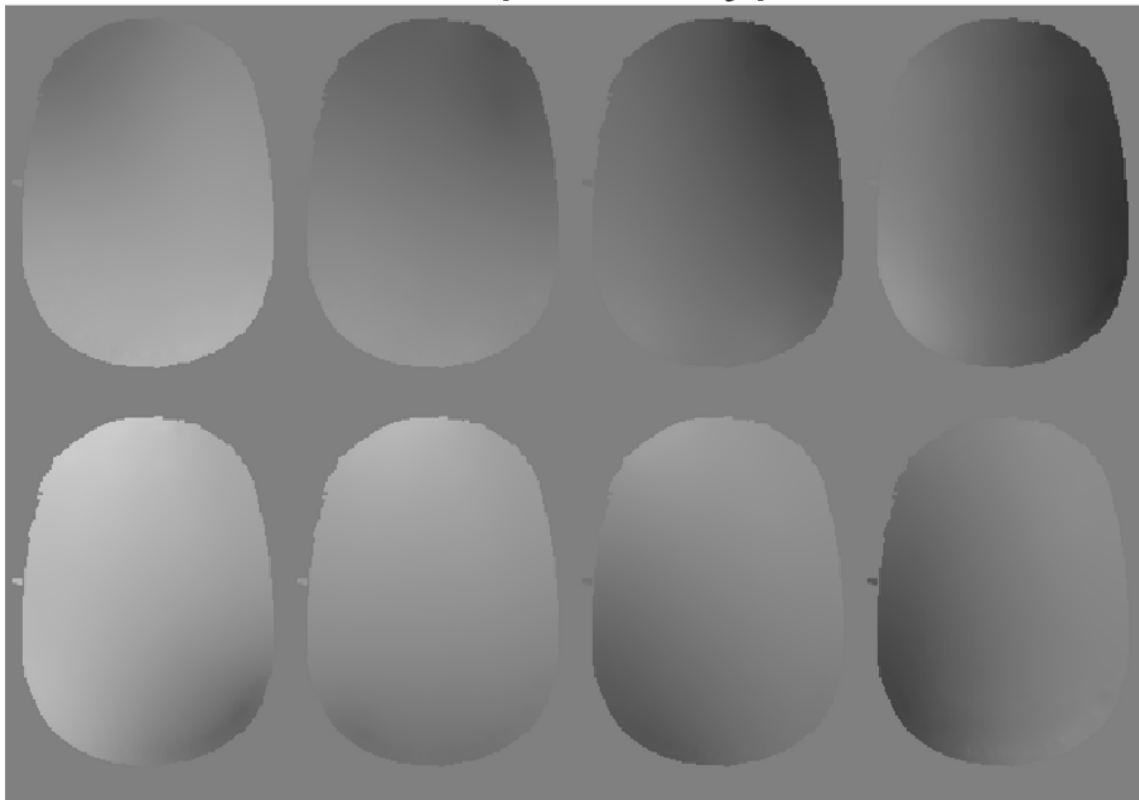


Figure 7: Phase of coil sensitivity map2.

Part 1.1: absolute sensitivity difference $|\text{map2} - \text{map1}|$

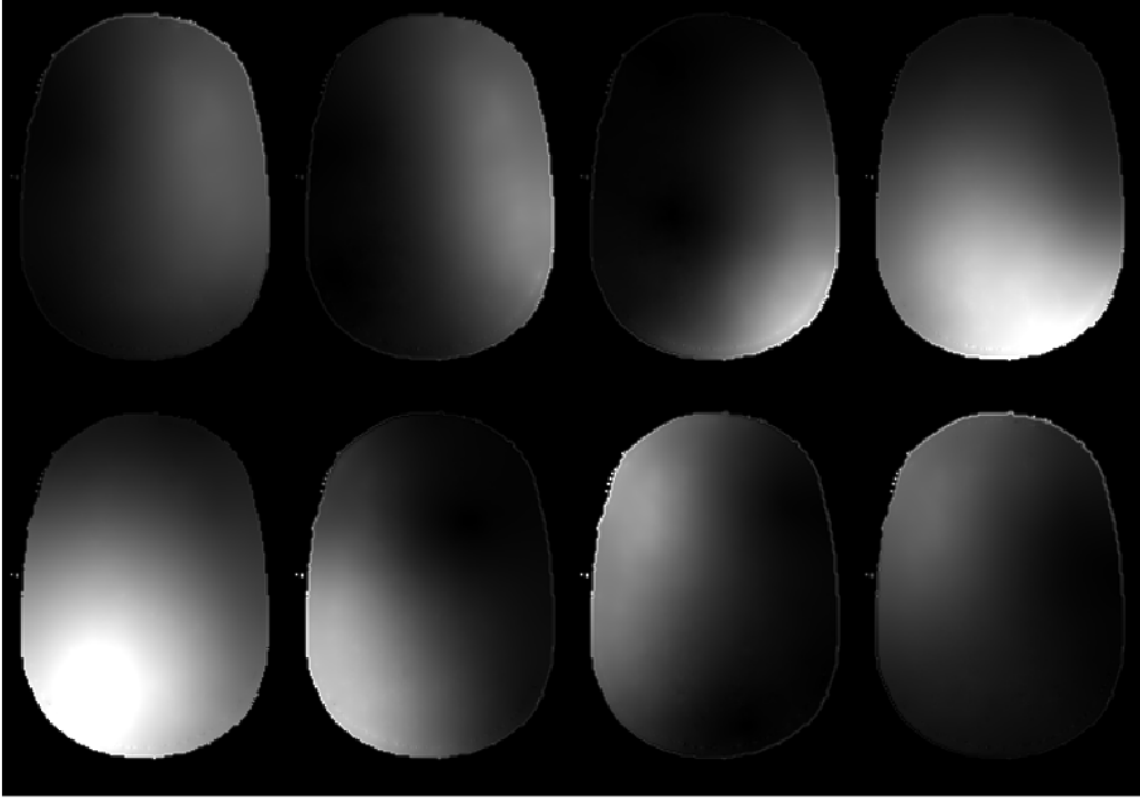


Figure 8: Absolute sensitivity-map difference $|\text{map2} - \text{map1}|$.

Code

```

1 M = fft2c(im);
2
3 show_montage(abs(im), [], 'Part_1.1:coil_image_magnitude', ...
4     fullfile(figDir, 'part1_1_coil_image_magnitude.png'));
5 show_montage(angle(im), [-pi pi], 'Part_1.1:coil_image_phase', ...
6     fullfile(figDir, 'part1_1_coil_image_phase.png'));
7 show_montage(kspace_spectrum(M), [], 'Part_1.1:coil_k-space_spectra', ...
8     fullfile(figDir, 'part1_1_coil_kspace_spectrum.png'));
9
10 show_montage(abs(map1), [], 'Part_1.1:map1_sensitivity_magnitude', ...
11     fullfile(figDir, 'part1_1_map1_magnitude.png'));
12 show_montage(angle(map1), [-pi pi], 'Part_1.1:map1_sensitivity_phase', ...
13     fullfile(figDir, 'part1_1_map1_phase.png'));
14
15 show_montage(abs(map2), [], 'Part_1.1:map2_sensitivity_magnitude', ...
16     fullfile(figDir, 'part1_1_map2_magnitude.png'));
17 show_montage(angle(map2), [-pi pi], 'Part_1.1:map2_sensitivity_phase', ...
18     fullfile(figDir, 'part1_1_map2_phase.png'));
19
20 map_diff = abs(map2 - map1);
21 show_montage(map_diff, [], 'Part_1.1:absolute_sensitivity_difference_map2-map1', ...
22     fullfile(figDir, 'part1_1_map_difference.png'));
23 fprintf('Part_1.1:mean_map2-map1=%.6g\n', mean(map_diff(:)));

```

1.2 OLC Reconstruction Using map1

For independent unit-variance coil noise, the optimally weighted linear combination is

$$\hat{m} = \frac{\sum_l c_l^* m_l}{\sum_l |c_l|^2}$$

The result from `map1`, after normalization by the 95th percentile intensity, is used as the reference image for Parts I and II.

Part 1.2: OLC magnitude using map1

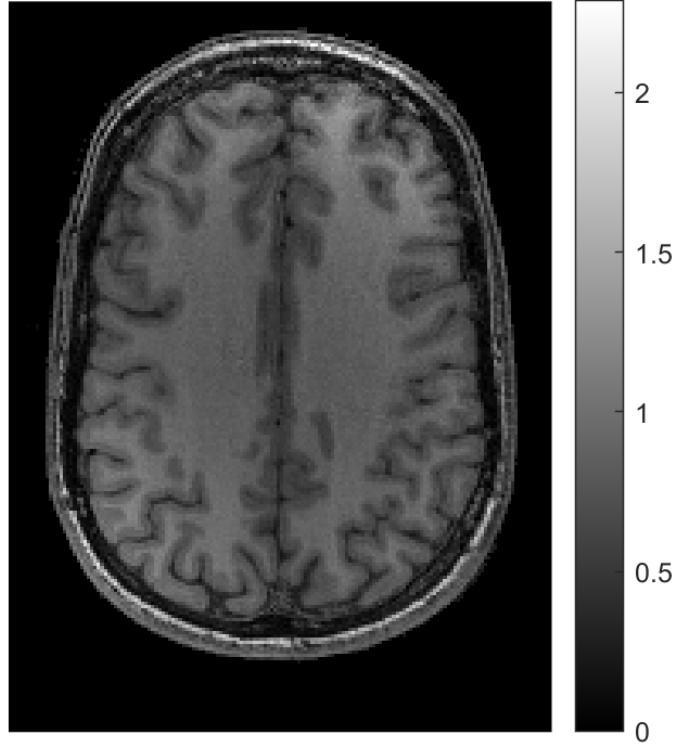


Figure 9: OLC reconstruction using map1.

MATLAB output: The reference maximum value for Parts I-II is `max_val = 2.28805`.

Code

```
1 olc_map1 = normalize95(abs(olc_recon(im, map1)));
2 max_val_sense = max(olc_map1(:));
3 ref_sense = olc_map1;
4
5 show_single_image(olc_map1, [0 max_val_sense], ...
6     'Part_1.2: OLC magnitude using map1', ...
7     fullfile(figDir, 'part1_2_olc_map1.png'));
8
9 fprintf('Part_1.2: max_val for Parts I-II = %.6g\n', max_val_sense);
```

1.3 OLC Reconstruction Using map2

Using `map2` changes the coil-combination weights. The reconstruction is still recognizable, but the PSNR decreases to 18.6567 dB relative to the `map1` OLC reference. The error is spatially

structured because the sensitivity-map mismatch varies across the image rather than acting like spatially uniform noise.

Part 1.3: OLC using map2

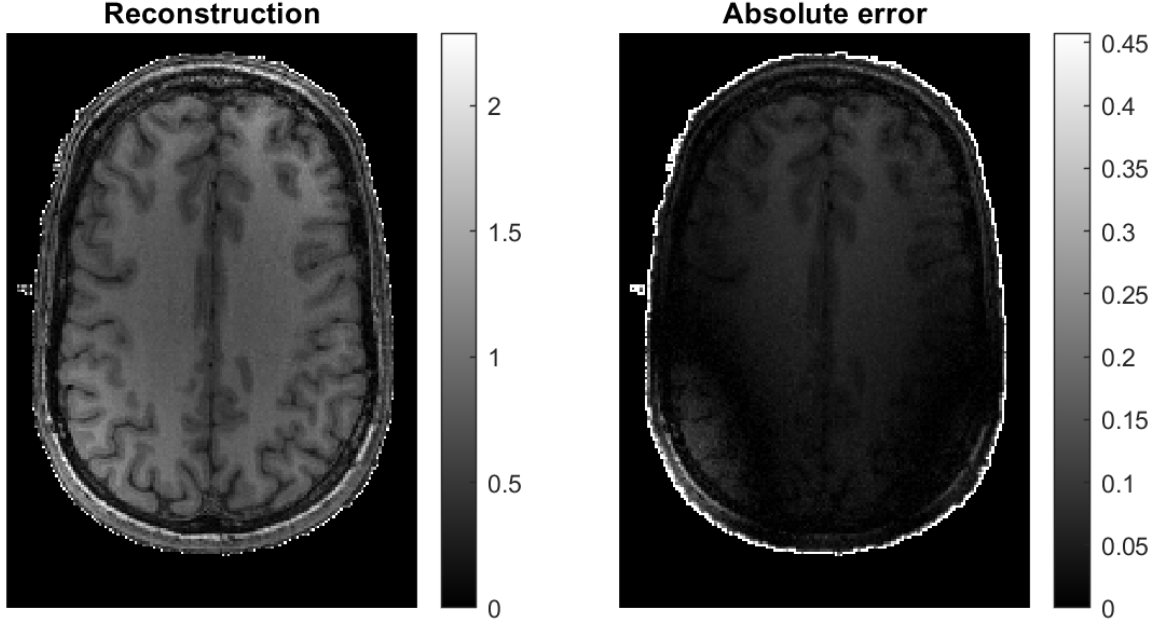


Figure 10: OLC reconstruction using map2 and absolute error relative to the map1 OLC reference.

MATLAB output: PSNR = 18.6567 dB, SSIM = 0.884596.

Code

```
1 olc_map2 = normalize95(abs(olc_recon(im, map2)));
2 metrics_13 = compute_metrics(olc_map2, ref_sense, max_val_sense);
3
4 show_image_and_error(olc_map2, ref_sense, max_val_sense, ...
5     'Part_1.3:_OLC_using_map2', ...
6     fullfile(figDir, 'part1_3_olc_map2_and_error.png'));
7
8 print_metrics('Part_1.3:_OLC_map2', metrics_13);
```

1.4 SoS Reconstruction

The square-root sum-of-squares reconstruction is

$$\hat{m}_{\text{SoS}} = \sqrt{\sum_l |m_l|^2}.$$

SoS does not require coil sensitivity maps, which makes it robust and simple. However, it retains coil sensitivity shading and intensity nonuniformity. The anatomy is visually recognizable, but the intensity profile differs from the OLC reference, which explains why SSIM can be reduced even when the PSNR is reasonable.

Part 1.4: SoS reconstruction

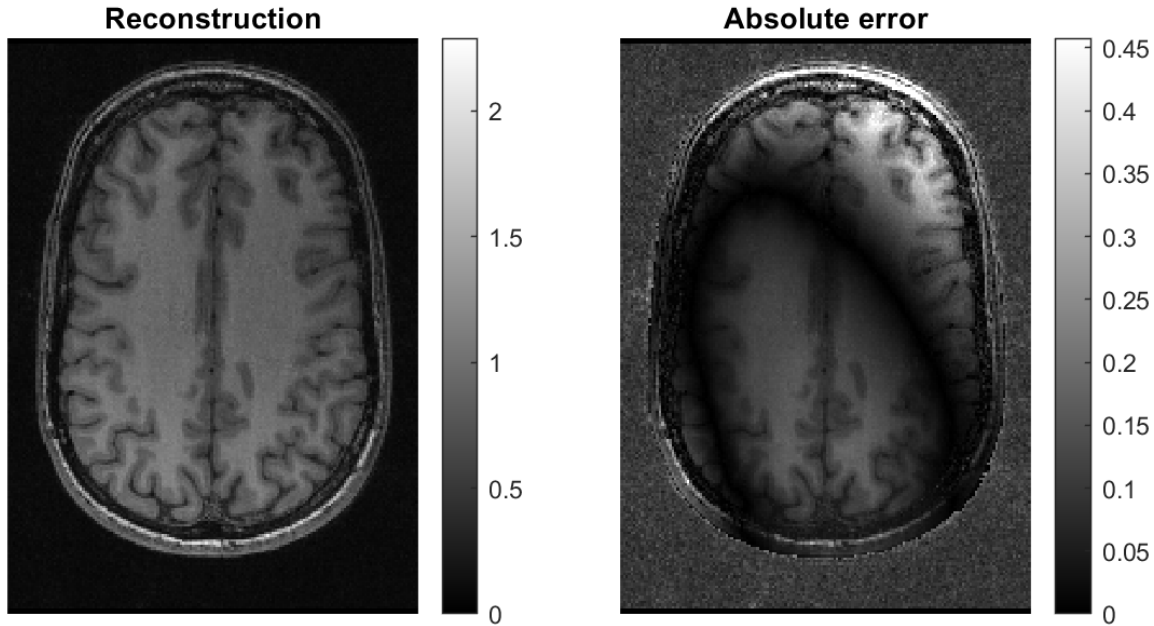


Figure 11: SoS reconstruction and absolute error relative to the OLC reference.

MATLAB output: PSNR = 25.6636 dB, SSIM = 0.658325.

Code

```
1 sos_part1 = normalize95(sos_combine(im));
2 metrics_14 = compute_metrics(sos_part1, ref_sense, max_val_sense);
3
4 show_image_and_error(sos_part1, ref_sense, max_val_sense, ...
5     'Part 1.4: SoS reconstruction', ...
6     fullfile(figDir, 'part1_4_sos_and_error.png'));
7
8 print_metrics('Part 1.4 SoS', metrics_14);
```

Part II – SENSE

2.1 Undersampled Images

The undersampling operation decimates k-space and then applies the inverse centered FFT. In the image domain this produces aliasing: voxels from different spatial locations fold onto the same measured pixel. The k-space spectra show the retained decimated sampling grid. For $R_x = 1, R_y = 2$, every other phase-encode line is retained; for $R_x = 1, R_y = 4$, only one out of four columns is retained; and for $R_x = 2, R_y = 2$, decimation occurs in both directions. The aliasing becomes more severe as the total acceleration factor increases.

Part 2.1: aliased images, $R_x=1$, $R_y=2$

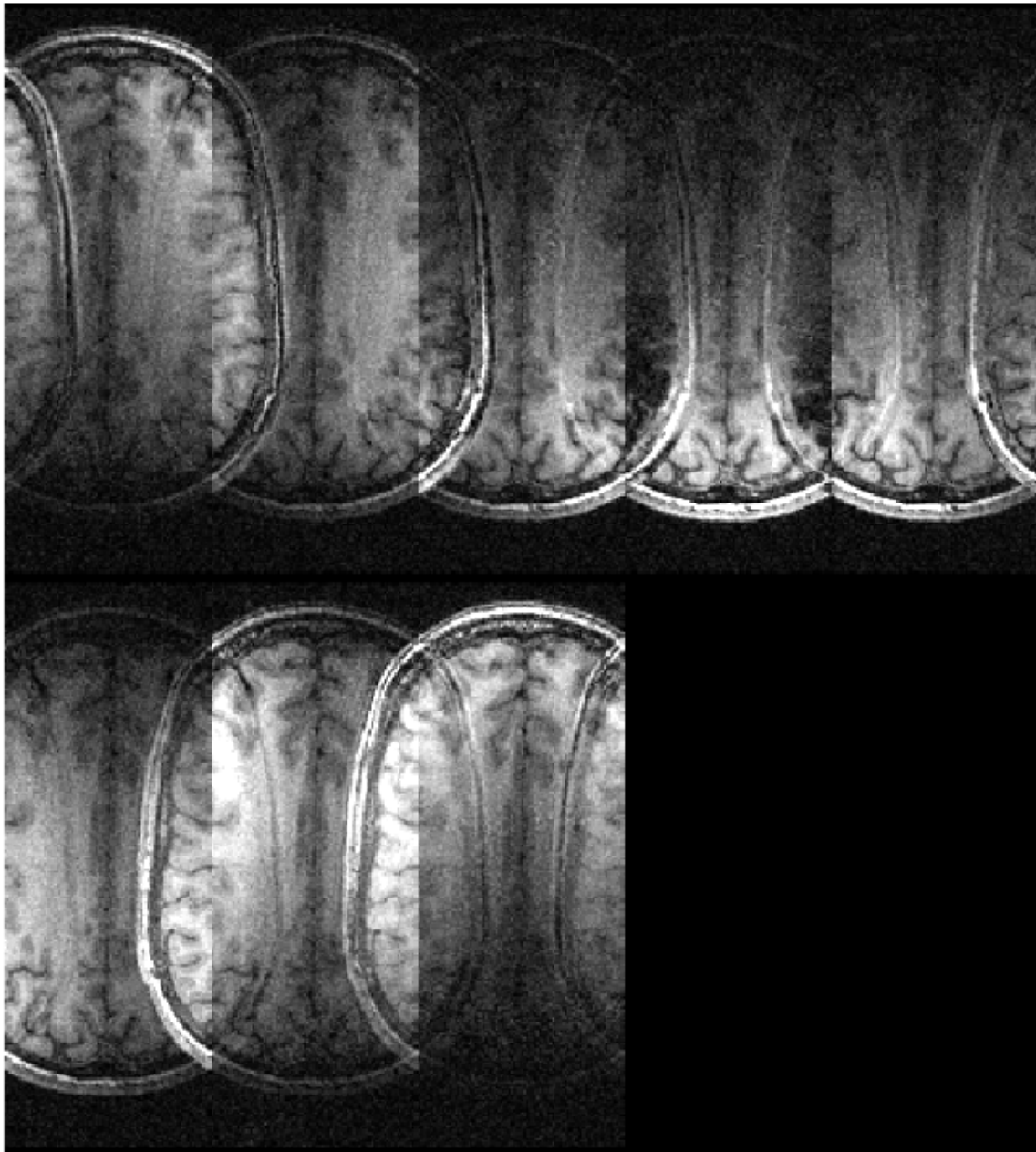


Figure 12: Aliased coil images for $R_x = 1$, $R_y = 2$.

Part 2.1: undersampled k-space, $R_x=1$, $R_y=2$

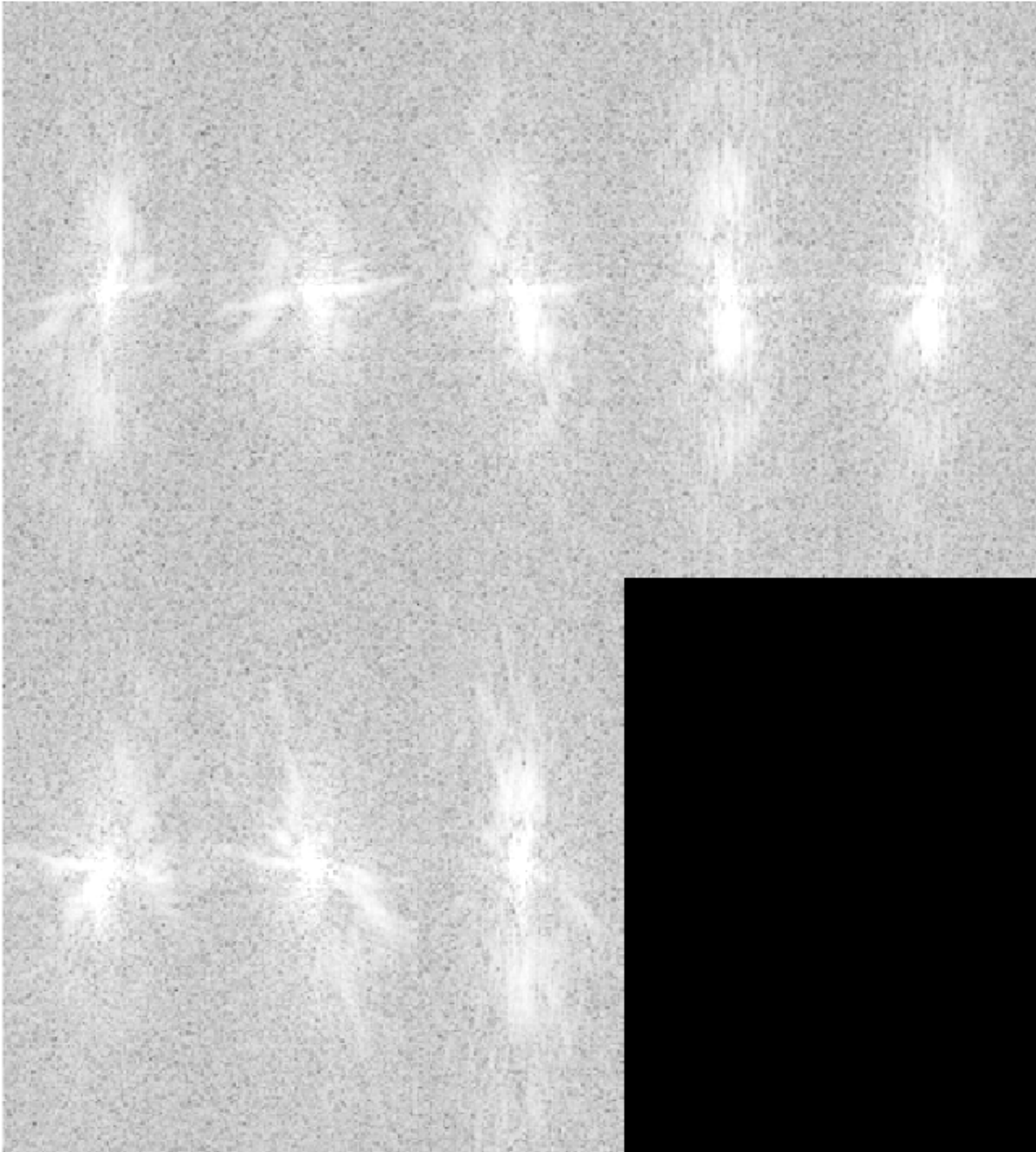


Figure 13: Undersampled k-space spectra for $R_x = 1$, $R_y = 2$.

Part 2.1: aliased images, $R_x=1$, $R_y=4$

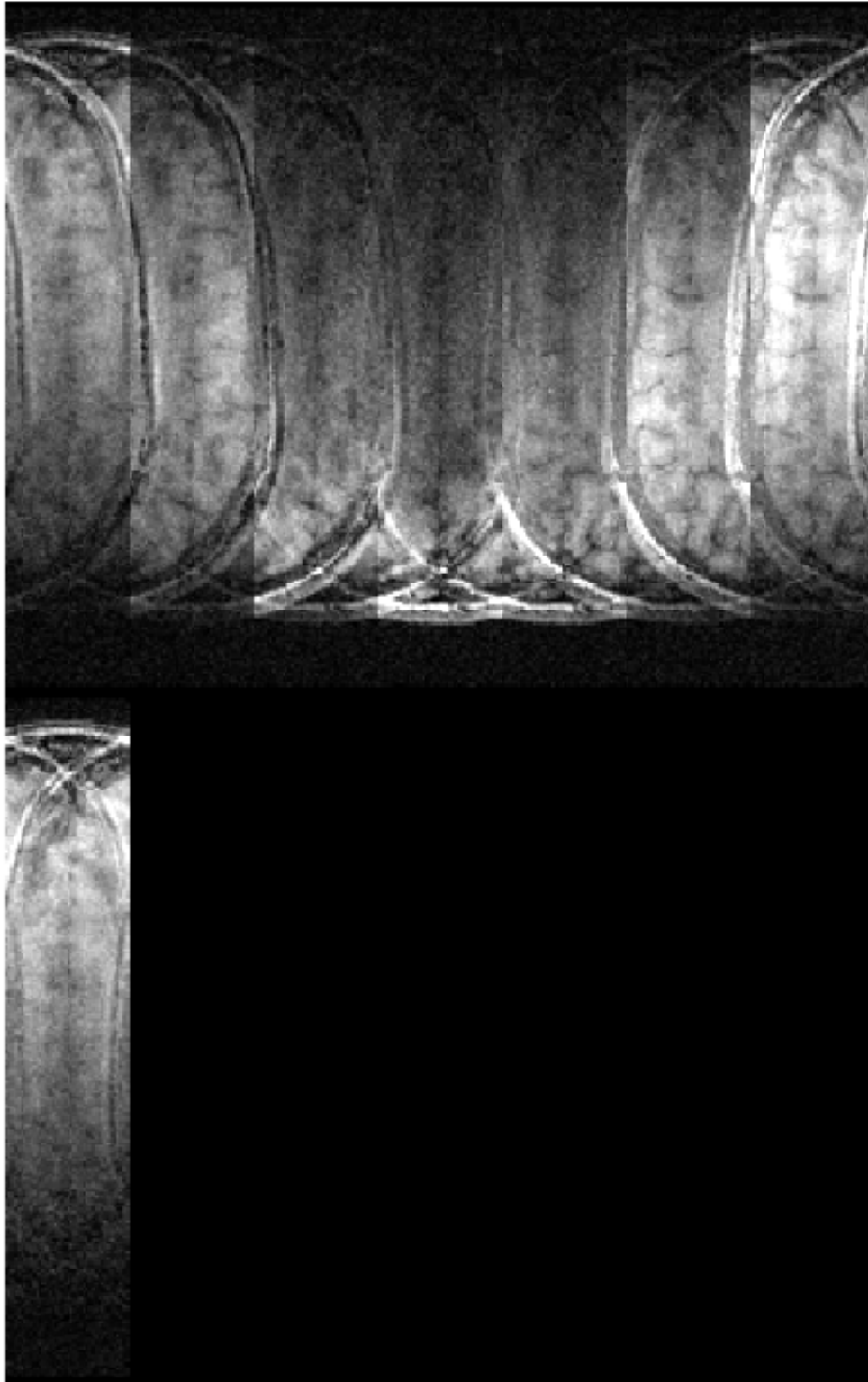


Figure 14: Aliased coil images for $R_x = 1$, $R_y = 4$.

Part 2.1: undersampled k-space, $R_x=1$, $R_y=4$

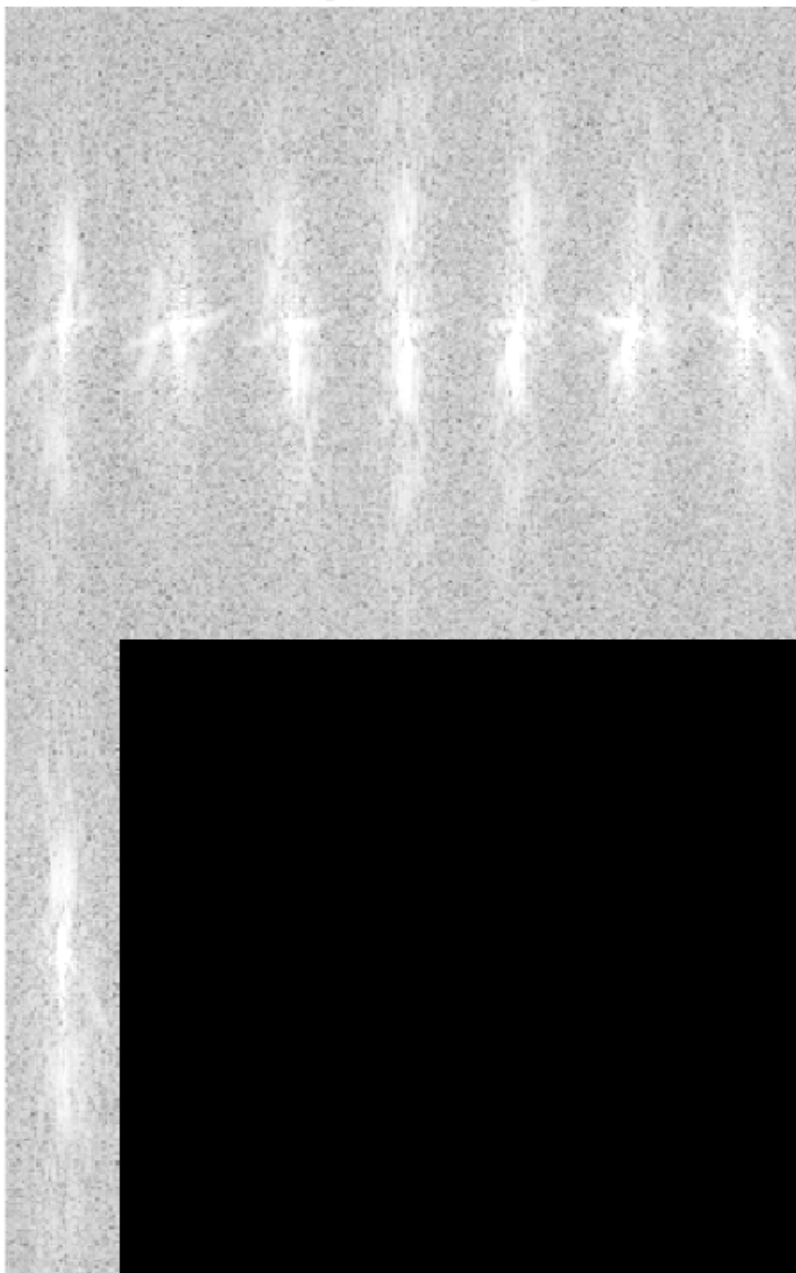


Figure 15: Undersampled k-space spectra for $R_x = 1$, $R_y = 4$.

Part 2.1: aliased images, $R_x=2$, $R_y=2$

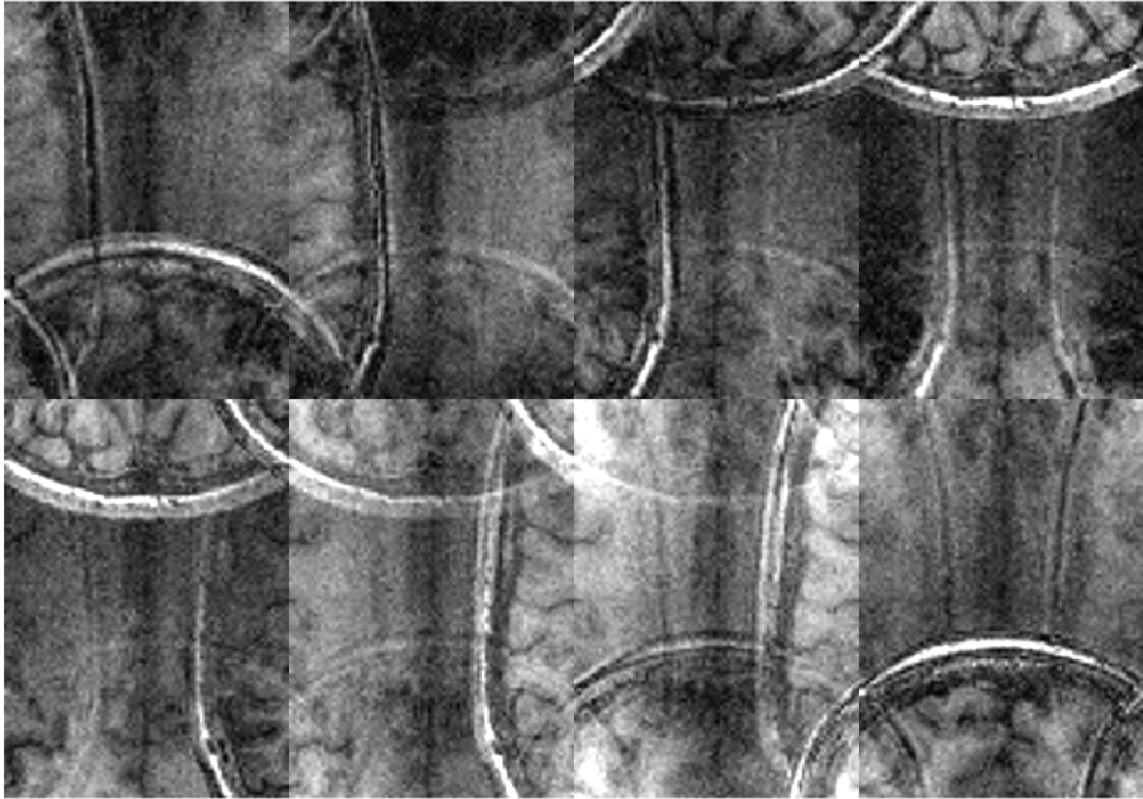


Figure 16: Aliased coil images for $R_x = 2$, $R_y = 2$.

Part 2.1: undersampled k-space, $R_x=2$, $R_y=2$

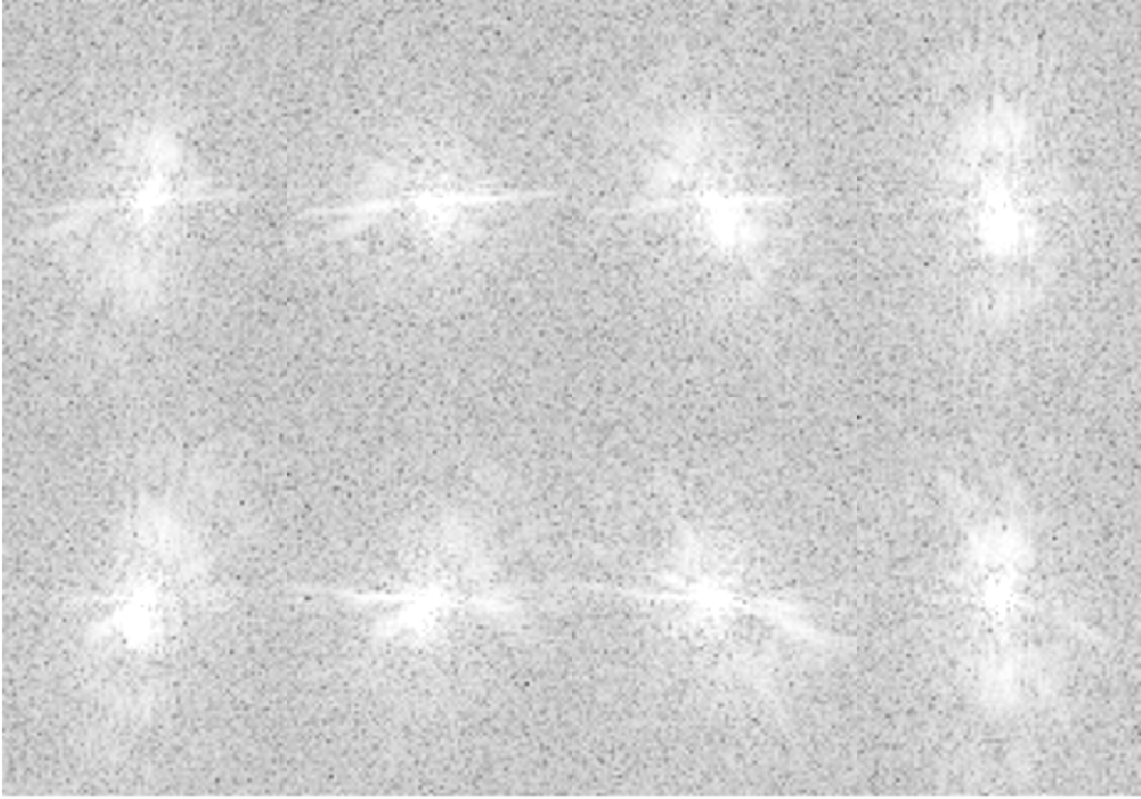


Figure 17: Undersampled k-space spectra for $R_x = 2$, $R_y = 2$.

Code

```
1 undersampled_tests = struct();
2 for caseIdx = 1:size(sense_cases, 1)
3     Rx = sense_cases(caseIdx, 1);
4     Ry = sense_cases(caseIdx, 2);
5     [imu, Mu] = undersample(im, Rx, Ry);
6
7     fieldName = sprintf('Rx%d_Ry%d', Rx, Ry);
8     undersampled_tests.(fieldName).imu = imu;
9     undersampled_tests.(fieldName).Mu = Mu;
10
11     show_montage(abs(imu), [], sprintf('Part_2.1:_%s', fieldName), Rx, Ry),
12     ...
13     fullfile(figDir, sprintf('part2_1_images_%s.png', Rx, Ry)));
14     show_montage(kspace_spectrum(Mu), [], sprintf('Part_2.1:_%s', fieldName), Rx, Ry), ...
15     fullfile(figDir, sprintf('part2_1_kspace_%s.png', Rx, Ry)));
end
```

2.2 SENSE Sanity Check

For $R_x = R_y = 1$ and $\lambda = 0$, SENSE reduces to the same voxelwise OLC reconstruction used in Part 1.2. Therefore PSNR should be extremely large and SSIM should be essentially 1, up to numerical

precision.

Part 2.2: SENSE sanity check, $R_x=1$, $R_y=1$, $\lambda=0$

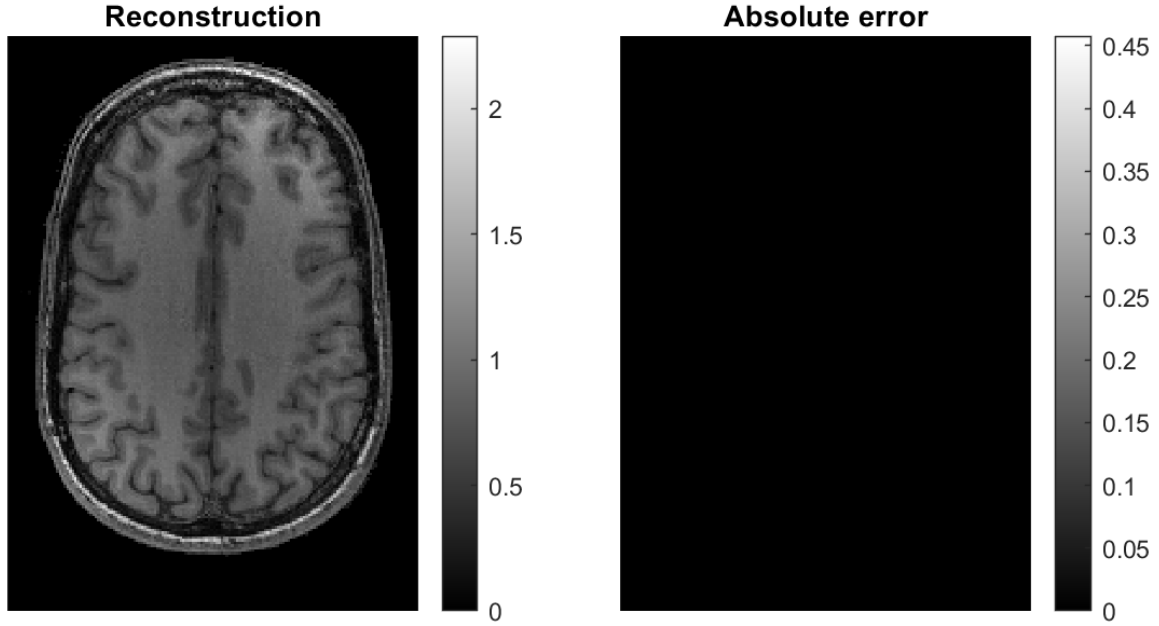


Figure 18: SENSE reconstruction for $R_x = R_y = 1$, $\lambda = 0$.

MATLAB output: PSNR = 321.4636 dB, SSIM = 1.000000.

Code

```

1 [imu_11, ~] = undersample(im, 1, 1);
2 im_sense_11 = l2sense(imu_11, map1, 1, 1, 0);
3 metrics_22 = compute_metrics(im_sense_11, ref_sense, max_val_sense);
4
5 show_image_and_error(im_sense_11, ref_sense, max_val_sense, ...
6     'Part_2.2:_SENSE_sanity_check,_Rx=1,_Ry=1,_lambda=0', ...
7     fullfile(figDir, 'part2_2_sense_Rx1_Ry1_lambda0.png'));
8
9 print_metrics('Part_2.2:_SENSE_Rx=1_Ry=1_lambda=0', metrics_22);

```

2.3 Unregularized SENSE

With acceleration, each aliased measurement corresponds to multiple source voxels. Without regularization, the inversion can become poorly conditioned, causing artifact and noise amplification. After correcting the centered aliasing indices, the $R_x = 1, R_y = 2$ and $R_x = 2, R_y = 2$ cases are reconstructed relatively well. However, the error images still show residual spatially varying error and noise amplification. The $R_x = 1, R_y = 4$ case remains the most difficult case because four source voxels must be unfolded from each aliased voxel.

Part 2.3: SENSE $R_x=1$, $R_y=2$, $\lambda=0$

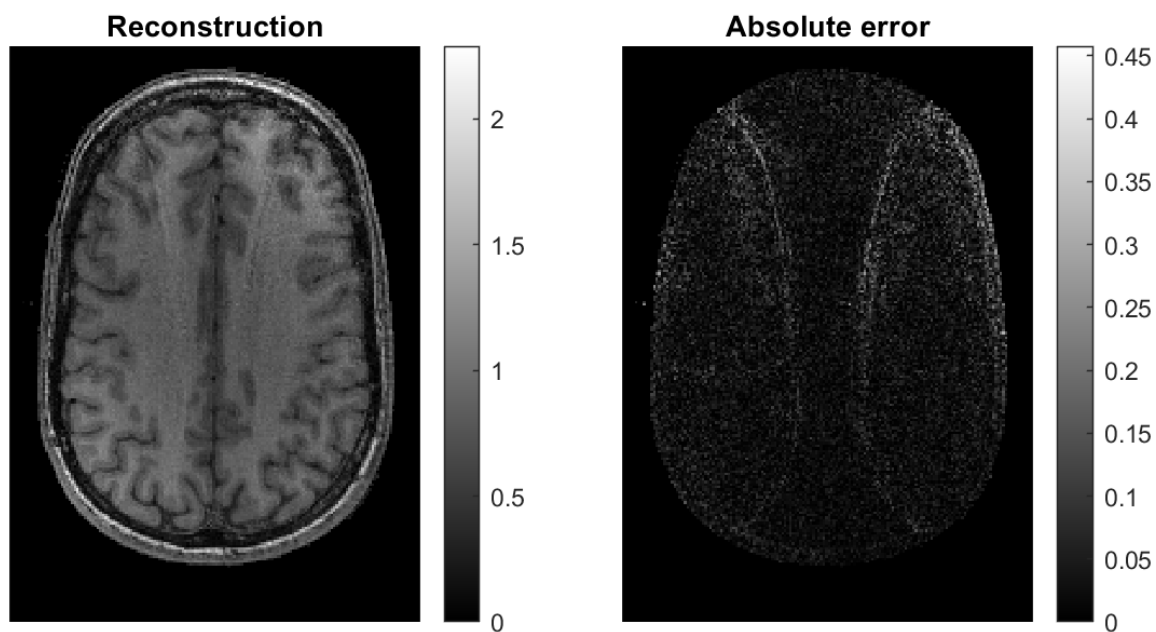


Figure 19: Unregularized SENSE for $R_x = 1$, $R_y = 2$.

MATLAB output: PSNR = 34.7076 dB, SSIM = 0.954200.

Part 2.3: SENSE $R_x=1$, $R_y=4$, $\lambda=0$

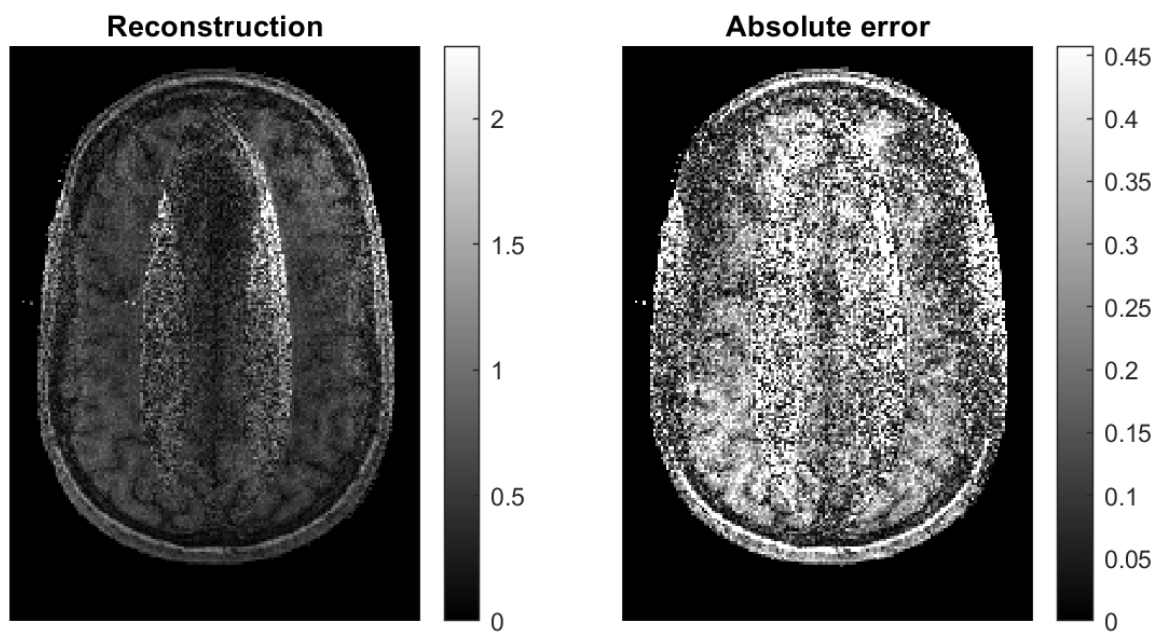


Figure 20: Unregularized SENSE for $R_x = 1$, $R_y = 4$.

MATLAB output: PSNR = 18.5253 dB, SSIM = 0.643093.

Part 2.3: SENSE Rx=2, Ry=2, lambda=0

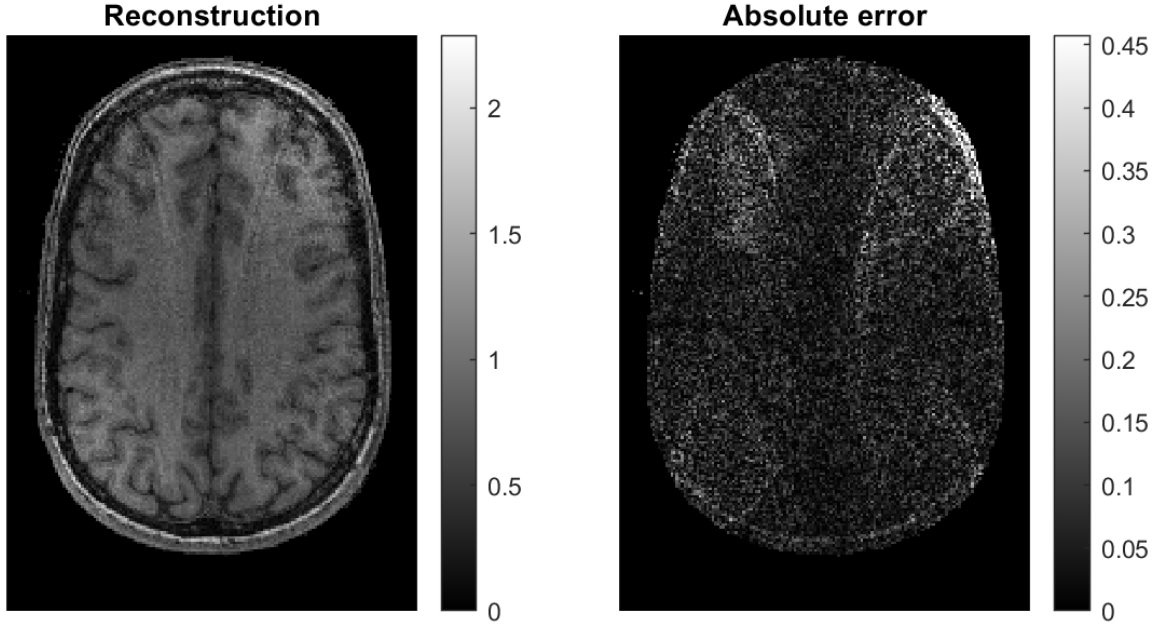


Figure 21: Unregularized SENSE for $R_x = 2$, $R_y = 2$.

MATLAB output: PSNR = 29.7818 dB, SSIM = 0.894558.

Code

```

1 sense_unreg = struct();
2 for caseIdx = 1:size(sense_cases, 1)
3     Rx = sense_cases(caseIdx, 1);
4     Ry = sense_cases(caseIdx, 2);
5     [imu, ~] = undersample(im, Rx, Ry);
6
7     im_sense = l2sense(imu, map1, Rx, Ry, 0);
8     metrics = compute_metrics(im_sense, ref_sense, max_val_sense);
9
10    fieldName = sprintf('Rx%d_Ry%d', Rx, Ry);
11    sense_unreg.(fieldName).image = im_sense;
12    sense_unreg.(fieldName).metrics = metrics;
13
14    show_image_and_error(im_sense, ref_sense, max_val_sense, ...
15        sprintf('Part_2.3:SENSE_Rx=%d,Ry=%d,lambda=0', Rx, Ry), ...
16        fullfile(figDir, sprintf('part2_3_sense_Rx%d_Ry%d_lambda0.png', Rx, Ry)));
17
18    print_metrics(sprintf('Part_2.3SENSE_Rx=%d,Ry=%d,lambda=0', Rx, Ry), metrics);
19 end

```

2.4 Regularized SENSE

The regularized reconstruction solves

$$\hat{m} = (C^H C + \lambda I)^{-1} C^H m_u.$$

Increasing λ improves conditioning and suppresses artifact/noise amplification, but too much regularization introduces bias and can blur or suppress true image detail. I searched over $\lambda \in \{0, 10^{-8}, \dots, 10^4\}$ and printed all PSNR/SSIM values in a table. To make the choice automatic, I used a combined score computed as the average of min-max normalized PSNR and min-max normalized SSIM values across the tested λ values. The selected λ is the row with the highest combined score.

MATLAB output: The code printed the full lambda table for each acceleration case and chose the row with the highest combined score. The selected value was $\lambda = 1$ for all three acceleration cases. For $R_x = 1, R_y = 4$, $\lambda = 10$ gave a slightly higher SSIM than $\lambda = 1$, but it also reduced PSNR, so the combined PSNR/SSIM score still selected $\lambda = 1$.

R_x	R_y	selected λ	PSNR / SSIM
1	2	1	35.5545 dB / 0.957431
1	4	1	25.5280 dB / 0.771031
2	2	1	31.1732 dB / 0.903190

Table 1: Selected regularization weights for SENSE using the combined PSNR/SSIM score.

Part 2.4: regularized SENSE $R_x=1, R_y=2$, lambda=1

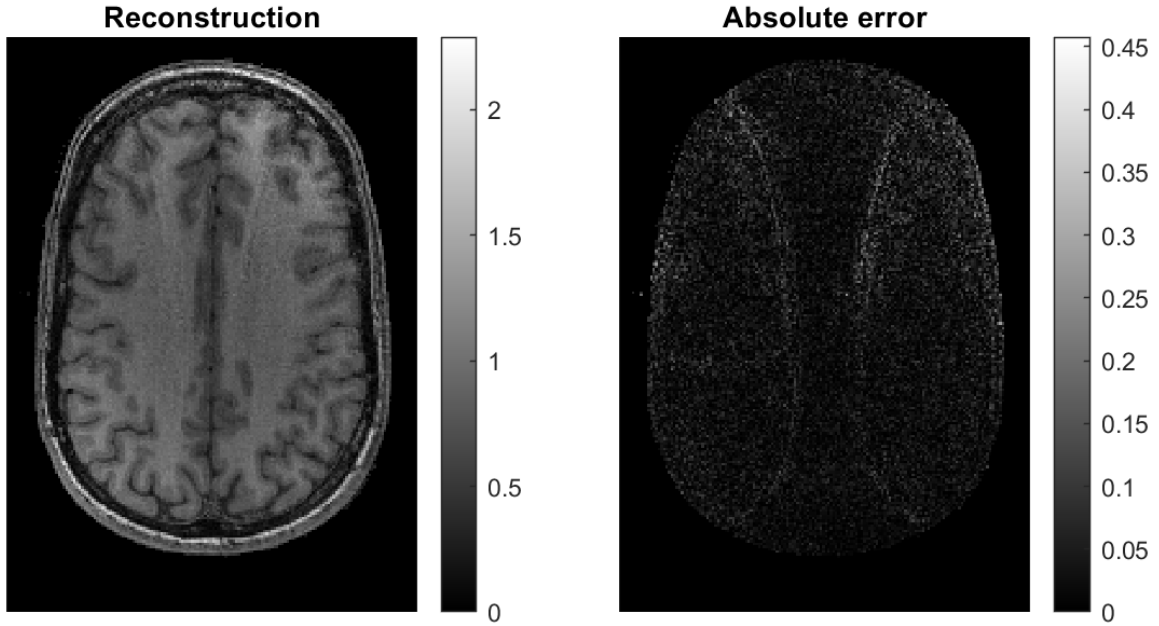


Figure 22: Regularized SENSE for $R_x = 1, R_y = 2$.

MATLAB output: $\lambda = 1$, PSNR = 35.5545 dB, SSIM = 0.957431.

Part 2.4: regularized SENSE $R_x=1$, $R_y=4$, $\lambda=1$

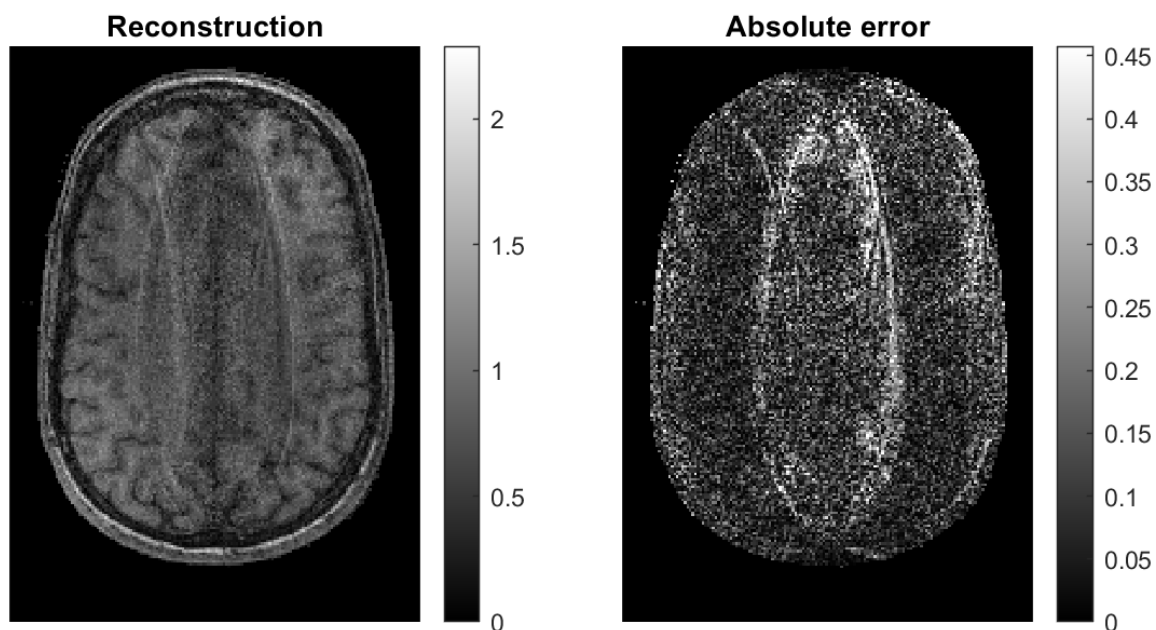


Figure 23: Regularized SENSE for $R_x = 1$, $R_y = 4$.

MATLAB output: $\lambda = 1$, PSNR = 25.5280 dB, SSIM = 0.771031.

Part 2.4: regularized SENSE $R_x=2$, $R_y=2$, $\lambda=1$

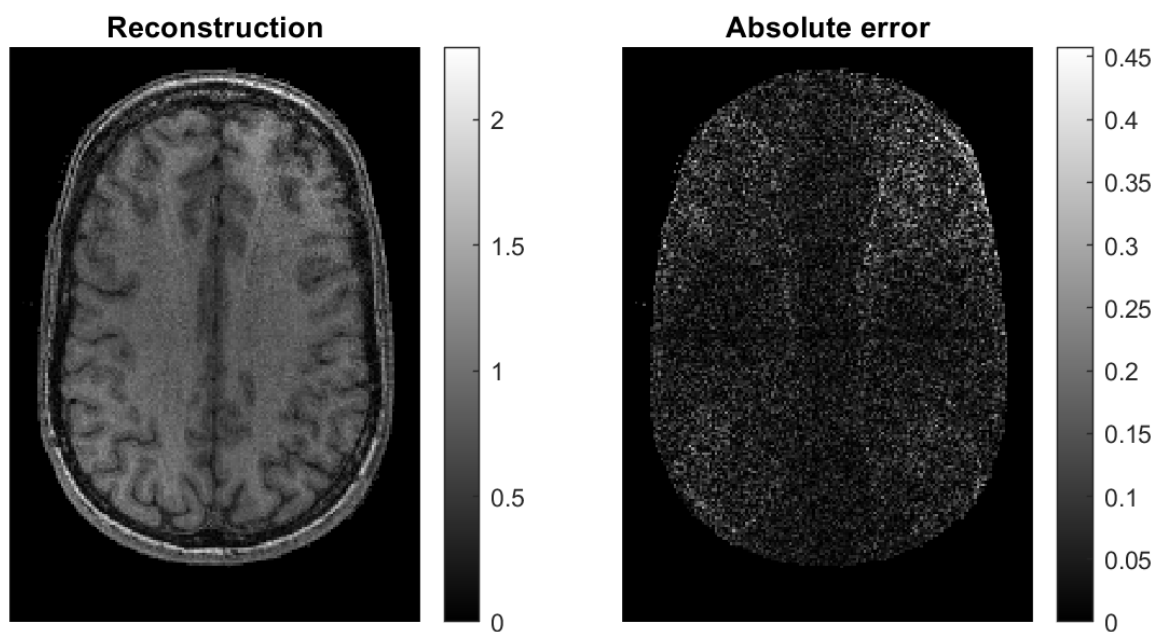


Figure 24: Regularized SENSE for $R_x = 2$, $R_y = 2$.

MATLAB output: $\lambda = 1$, PSNR = 31.1732 dB, SSIM = 0.903190.

Code

```

1 lambda_candidates = [0, 10.^(-8:4)];
2 lambda_sense = zeros(1, size(sense_cases, 1));
3 sense_reg = struct();
4 lambda_scores = struct();
5
6 for caseIdx = 1:size(sense_cases, 1)
7     Rx = sense_cases(caseIdx, 1);
8     Ry = sense_cases(caseIdx, 2);
9     [imu, ~] = undersample(im, Rx, Ry);
10
11     candidateImages = cell(numel(lambda_candidates), 1);
12     scoreRows = zeros(numel(lambda_candidates), 4);
13
14     for k = 1:numel(lambda_candidates)
15         lambda = lambda_candidates(k);
16         candidate = l2sense(imu, map1, Rx, Ry, lambda);
17         metrics = compute_metrics(candidate, ref_sense, max_val_sense);
18
19         candidateImages{k} = candidate;
20         scoreRows(k, 1:3) = [lambda, metrics.psnr, metrics.ssim];
21     end
22
23     psnrVals = scoreRows(:, 2);
24     ssimVals = scoreRows(:, 3);
25     psnrScore = (psnrVals - min(psnrVals)) ./ (max(psnrVals) - min(psnrVals) + eps);
26     ssimScore = (ssimVals - min(ssimVals)) ./ (max(ssimVals) - min(ssimVals) + eps);
27     scoreRows(:, 4) = 0.5 * psnrScore + 0.5 * ssimScore;
28
29     [~, bestIdx] = max(scoreRows(:, 4));
30     fieldName = sprintf('Rx%d_Ry%d', Rx, Ry);
31     bestLambda = scoreRows(bestIdx, 1);
32     bestImage = candidateImages{bestIdx};
33     bestMetrics = struct('psnr', scoreRows(bestIdx, 2), 'ssim', scoreRows(bestIdx, 3));
34
35     lambda_sense(caseIdx) = bestLambda;
36     sense_reg(fieldName).image = bestImage;
37     sense_reg(fieldName).lambda = bestLambda;
38     sense_reg(fieldName).metrics = bestMetrics;
39     sense_reg(fieldName).scores = scoreRows;
40     lambda_scores(fieldName) = array2table(scoreRows, ...
41         'VariableNames', {'lambda', 'PSNR_dB', 'SSIM', 'combined_score'});
42
43     show_image_and_error(bestImage, ref_sense, max_val_sense, ...
44         sprintf('Part_2.4:regularized_SENSE_Rx=%d,Ry=%d,lambda=%g', Rx, Ry, bestLambda
45         ), ...
46         fullfile(figDir, sprintf('part2_4_sense_Rx%d_Ry%d_selected.png', Rx, Ry)));
47
48     fprintf('\nPart_2.4_lambda_search_table_for_Rx=%dRy=%d:\n', Rx, Ry);
49     disp(lambda_scores(fieldName));
50     fprintf('Part_2.4_selection_criterion: highest_combined_score.\n');
51     fprintf('Part_2.4_Rx=%dRy=%d_selected_lambda=%g\n', Rx, Ry, bestLambda);
52     print_metrics(sprintf('Part_2.4_SENSE_Rx=%dRy=%d', Rx, Ry), bestMetrics);
53 end

```

2.5 Sensitivity to Coil Map Errors

Repeating SENSE with `map2` uses the same automatically selected regularization weights as Part 2.4. Since SENSE relies directly on the encoding matrix formed by the sensitivity maps, inaccurate maps lead to residual aliasing and spatially structured errors. The `map2` results are worse than the corresponding `map1` results in all three cases, confirming the sensitivity of SENSE to coil-map accuracy.

Part 2.5: SENSE with `map2` $R_x=1$, $R_y=2$, $\lambda=1$

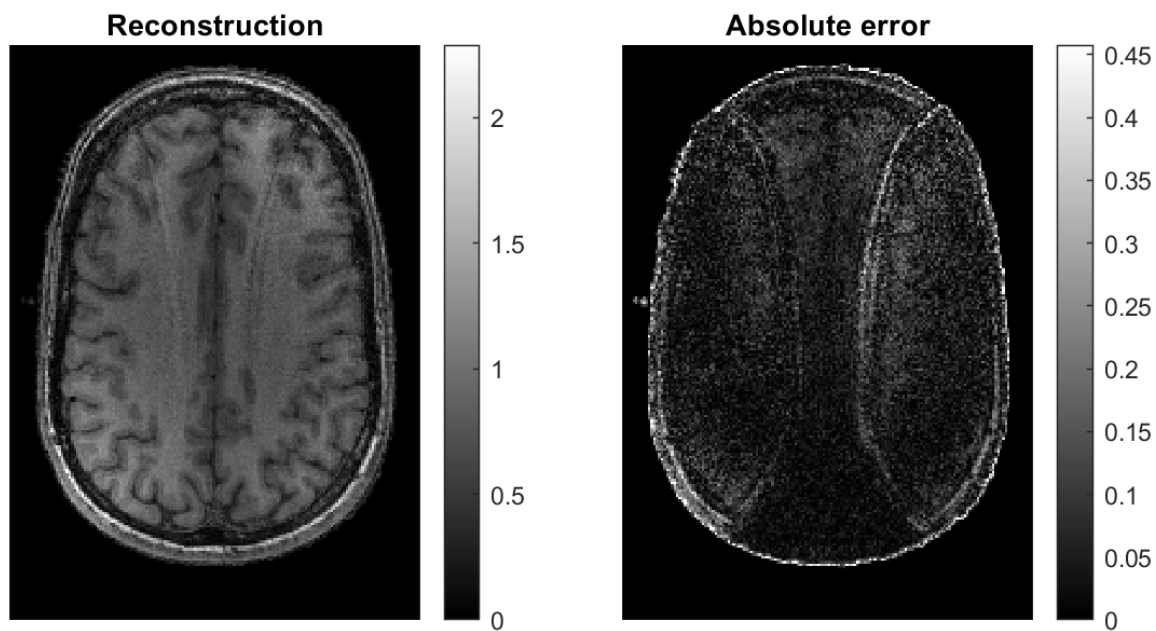


Figure 25: Regularized SENSE with `map2` for $R_x = 1$, $R_y = 2$.

MATLAB output: PSNR = 29.4891 dB, SSIM = 0.919559.

Part 2.5: SENSE with map2 $R_x=1$, $R_y=4$, $\lambda=1$

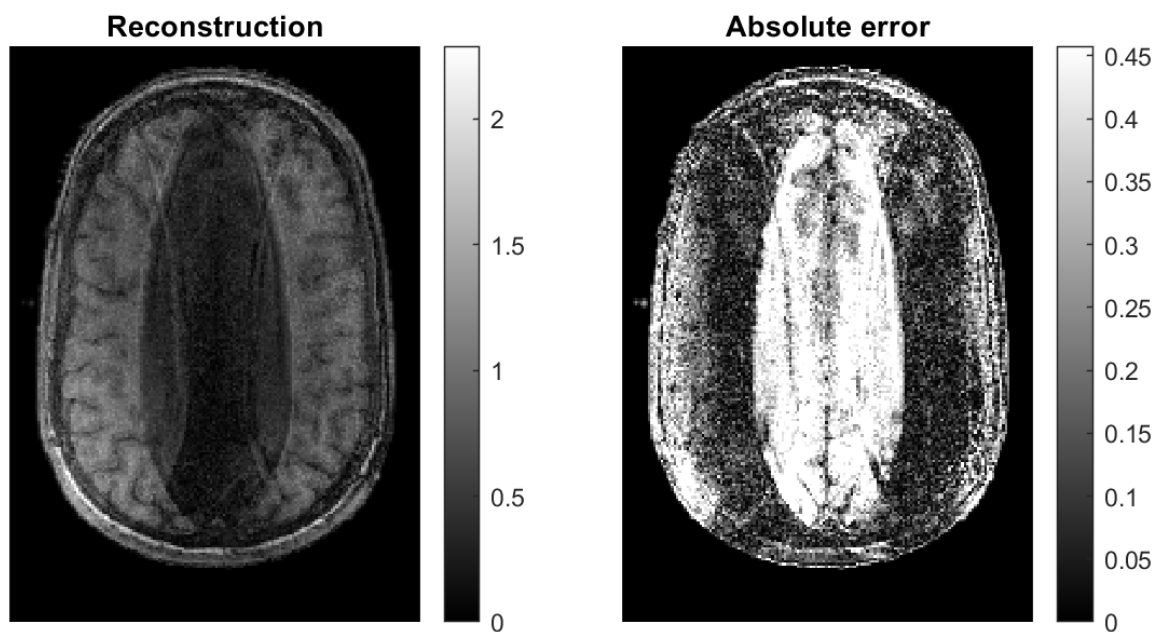


Figure 26: Regularized SENSE with map2 for $R_x = 1$, $R_y = 4$.

MATLAB output: PSNR = 19.0632 dB, SSIM = 0.700300.

Part 2.5: SENSE with map2 $R_x=2$, $R_y=2$, $\lambda=1$

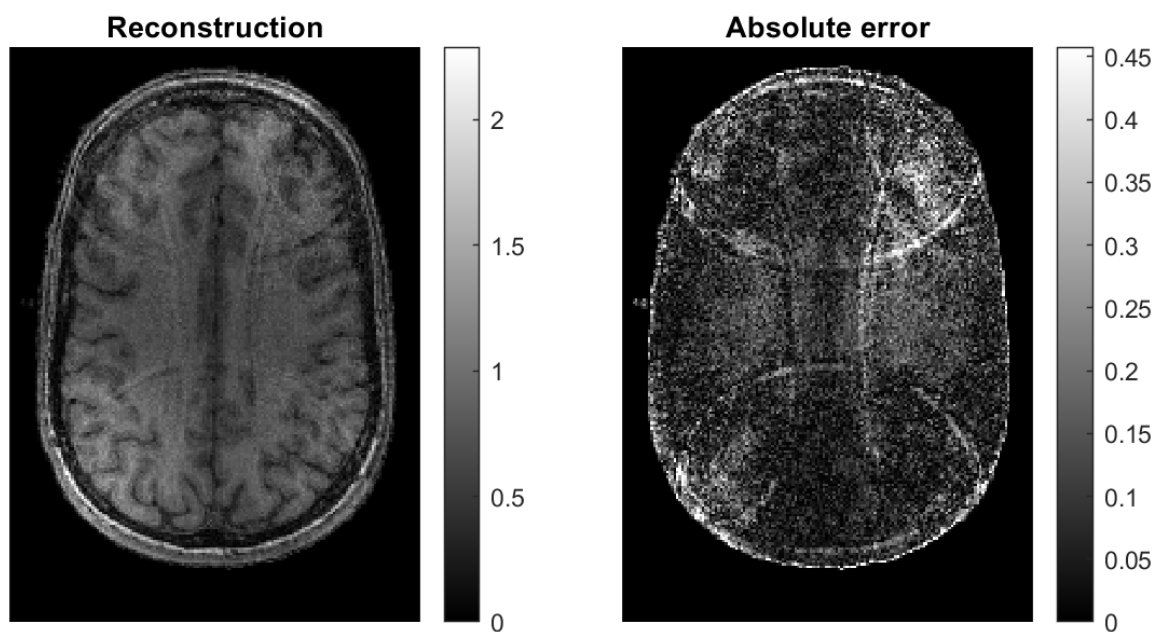


Figure 27: Regularized SENSE with map2 for $R_x = 2$, $R_y = 2$.

MATLAB output: PSNR = 26.0102 dB, SSIM = 0.852694.

Code

```
1 sense_map2 = struct();
2 for caseIdx = 1:size(sense_cases, 1)
3     Rx = sense_cases(caseIdx, 1);
4     Ry = sense_cases(caseIdx, 2);
5     lambda = lambda_sense(caseIdx);
6     [imu, ~] = undersample(im, Rx, Ry);
7
8     im_sense = l2sense(imu, map2, Rx, Ry, lambda);
9     metrics = compute_metrics(im_sense, ref_sense, max_val_sense);
10
11     fieldName = sprintf('Rx%d_Ry%d', Rx, Ry);
12     sense_map2.(fieldName).image = im_sense;
13     sense_map2.(fieldName).metrics = metrics;
14
15     show_image_and_error(im_sense, ref_sense, max_val_sense, ...
16         sprintf('Part_2.5:SENSE_map2_Rx=%d,Ry=%d,lambda=%g', Rx, Ry, lambda), ...
17         fullfile(figDir, sprintf('part2_5_sense_map2_Rx%d_Ry%d.png', Rx, Ry)));
18
19     print_metrics(sprintf('Part_2.5SENSE_map2_Rx=%d_Ry=%d', Rx, Ry), metrics);
20 end
```

2.6 g-Factor Derivation

For l2-regularized SENSE,

$$A = (C^H C + \lambda I)^{-1} C^H.$$

With $\Psi = I$,

$$\text{Cov}(\hat{m}) = A A^H = (C^H C + \lambda I)^{-1} C^H C ((C^H C + \lambda I)^{-1})^H.$$

Let $G = C^H C$ and $B = (G + \lambda I)^{-1}$. Then

$$\text{Cov}(\hat{m}) = B G B^H.$$

Using the same definition as the unregularized case, the regularized g-factor for source voxel i is

$$g_i = \sqrt{\text{Re}(G_{ii} [B G B^H]_{ii})}.$$

2.7 g-Factor Maps

The g-factor maps show where the SENSE encoding is poorly conditioned. Values near 1 indicate favorable conditioning, while larger values indicate stronger noise amplification. With the selected regularization values, the g-factor is kept bounded in most of the object. I also report the maximum and mean g-factor values inside the object mask, because these summarize the worst-case and average conditioning behavior. Higher acceleration generally produces larger and more spatially variable g-factors because more source voxels must be separated from the same coil measurements.

Part 2.7: g-factor $R_x=1$, $R_y=2$, $\lambda=1$

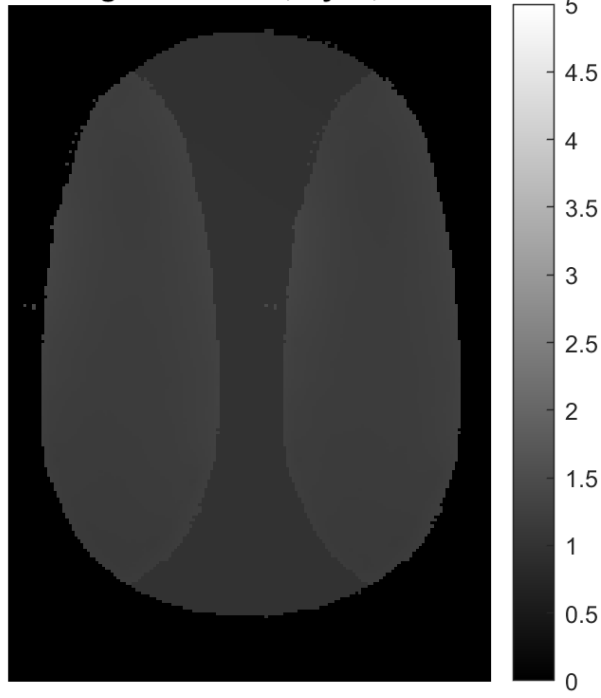


Figure 28: g-factor map for $R_x = 1$, $R_y = 2$.

Part 2.7: g-factor $R_x=1$, $R_y=4$, $\lambda=1$

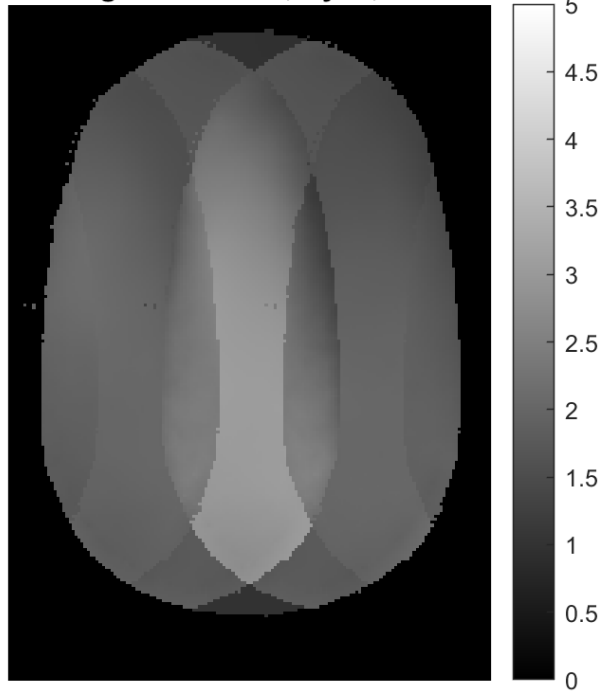


Figure 29: g-factor map for $R_x = 1$, $R_y = 4$.

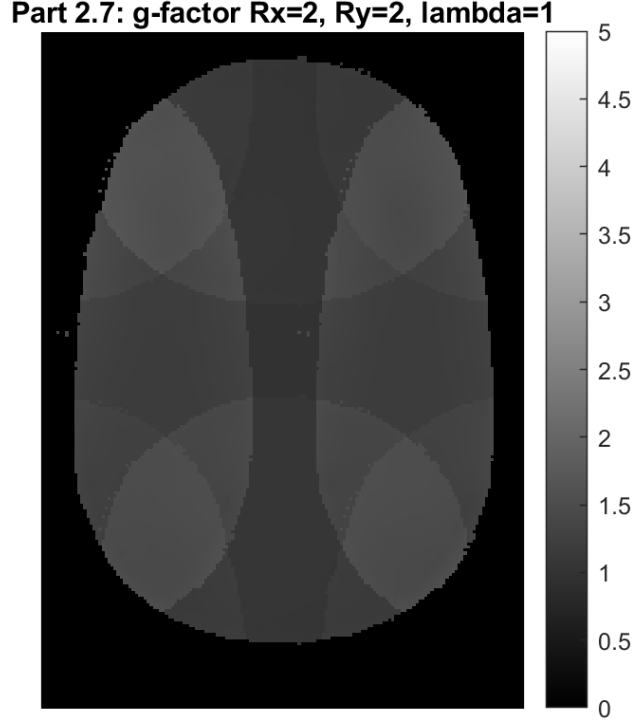


Figure 30: g-factor map for $R_x = 2$, $R_y = 2$.

MATLAB output: With the selected $\lambda = 1$ values, the printed g-factor statistics were:

R_x	R_y	maximum g-factor	mean g-factor
1	2	1.4491	1.1301
1	4	3.0937	2.0126
2	2	1.7196	1.2826

Table 2: g-factor statistics for regularized SENSE using the selected λ values.

Code

```

1 g_selected = struct();
2 g_selected_stats = struct();
3 for caseIdx = 1:size(sense_cases, 1)
4     Rx = sense_cases(caseIdx, 1);
5     Ry = sense_cases(caseIdx, 2);
6     lambda = lambda_sense(caseIdx);
7
8     g = gfactor(map1, Rx, Ry, lambda);
9     fieldName = sprintf('Rx%d_Ry%d', Rx, Ry);
10    g_selected.(fieldName) = g;
11    g_selected_stats.(fieldName) = gfactor_stats(g, map1);
12
13    show_single_image(g, [0 5], ...
14        sprintf('Part_2.7:g-factor_Rx=%d_Ry=%d_lambda=%g', Rx, Ry, lambda), ...
15        fullfile(figDir, sprintf('part2_7_gfactor_Rx%d_Ry%d.png', Rx, Ry)));
16
17    fprintf('Part_2.7:g-factor_Rx=%d_Ry=%d_lambda=%g: max=%.4f, mean=%.4f\n', ...

```

```

18         Rx, Ry, lambda, g_selected_stats.(fieldName).max, g_selected_stats.(fieldName).
           mean);
19 end

```

2.8 Insufficient Regularization

When the selected λ values are multiplied by 10^{-6} , the reconstruction approaches the unregularized case. The g-factor maps therefore show higher peaks and stronger spatial variation, especially in regions where the coil sensitivities do not separate aliased voxels well. This is most important for the more challenging high-acceleration case.

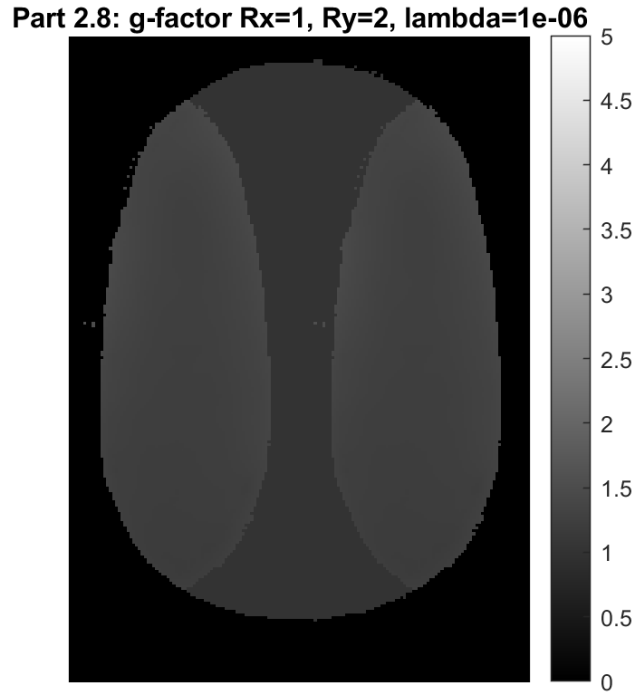


Figure 31: g-factor map with insufficient regularization for $R_x = 1$, $R_y = 2$.

Part 2.8: g-factor $R_x=1$, $R_y=4$, $\lambda=1e-06$

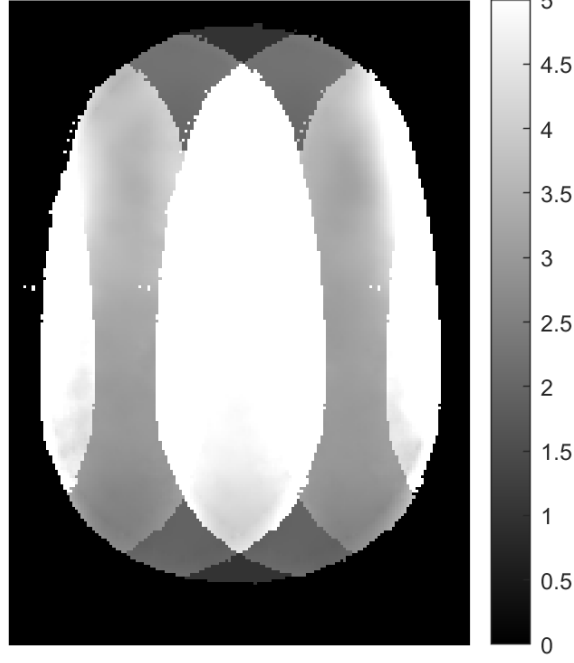


Figure 32: g-factor map with insufficient regularization for $R_x = 1$, $R_y = 4$.

Part 2.8: g-factor $R_x=2$, $R_y=2$, $\lambda=1e-06$

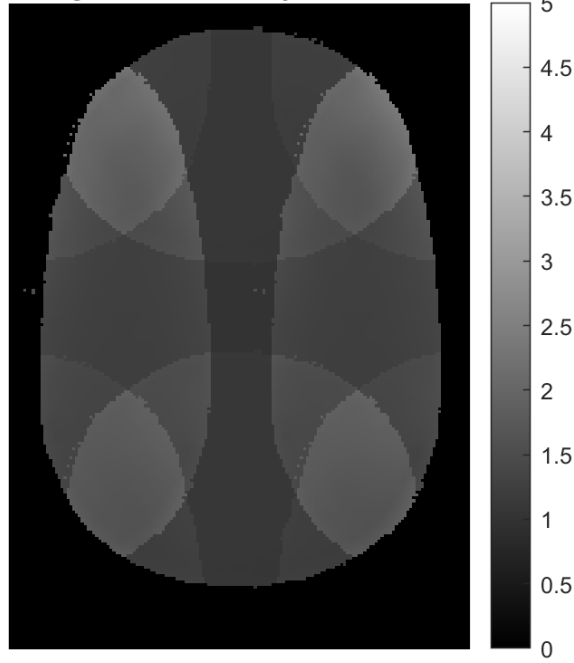


Figure 33: g-factor map with insufficient regularization for $R_x = 2$, $R_y = 2$.

MATLAB output: After multiplying the selected λ values by 10^{-6} , the printed g-factor statistics were:

R_x	R_y	maximum g-factor	mean g-factor
1	2	1.5605	1.1824
1	4	31.4601	5.8143
2	2	2.4122	1.4204

Table 3: g-factor statistics when the selected regularization weights are reduced by 10^{-6} .

Code

```

1 g_insufficient = struct();
2 g_insufficient_stats = struct();
3 for caseIdx = 1:size(sense_cases, 1)
4     Rx = sense_cases(caseIdx, 1);
5     Ry = sense_cases(caseIdx, 2);
6     lambda = lambda_sense(caseIdx) * 1e-6;
7
8     g = gfactor(map1, Rx, Ry, lambda);
9     fieldName = sprintf('Rx%d_Ry%d', Rx, Ry);
10    g_insufficient.(fieldName) = g;
11    g_insufficient_stats.(fieldName) = gfactor_stats(g, map1);
12
13    show_single_image(g, [0 5], ...
14        sprintf('Part_2.8_g-factor_Rx=%d_Ry=%d_lambda=%g', Rx, Ry, lambda), ...
15        fullfile(figDir, sprintf('part2_8_gfactor_Rx%d_Ry%d.png', Rx, Ry)));
16
17    fprintf('Part_2.8_g-factor_Rx=%d_Ry=%d_lambda=%g: max=%.4f, mean=%.4f\n', ...
18        Rx, Ry, lambda, g_insufficient_stats.(fieldName).max, g_insufficient_stats.(
19        fieldName).mean);
20 end

```

Part III – GRAPPA

For GRAPPA, the fully sampled SoS image after 95th percentile normalization is used as the reference. The MATLAB setup printed `max_val = 2.08358` for this Part III reference.

3.1 Calibration Images

The calibration region keeps only the central part of k-space. Small calibration regions produce very smooth and blurry images because only low spatial frequencies are retained. The corresponding k-space figures confirm that only the central square is preserved. As the calibration size increases from 8×8 to 32×32 , more anatomical detail appears. These low-frequency calibration images also contain smooth coil sensitivity variation, which is the information GRAPPA uses for autocalibration.

Part 3.1: calibration images 8x8

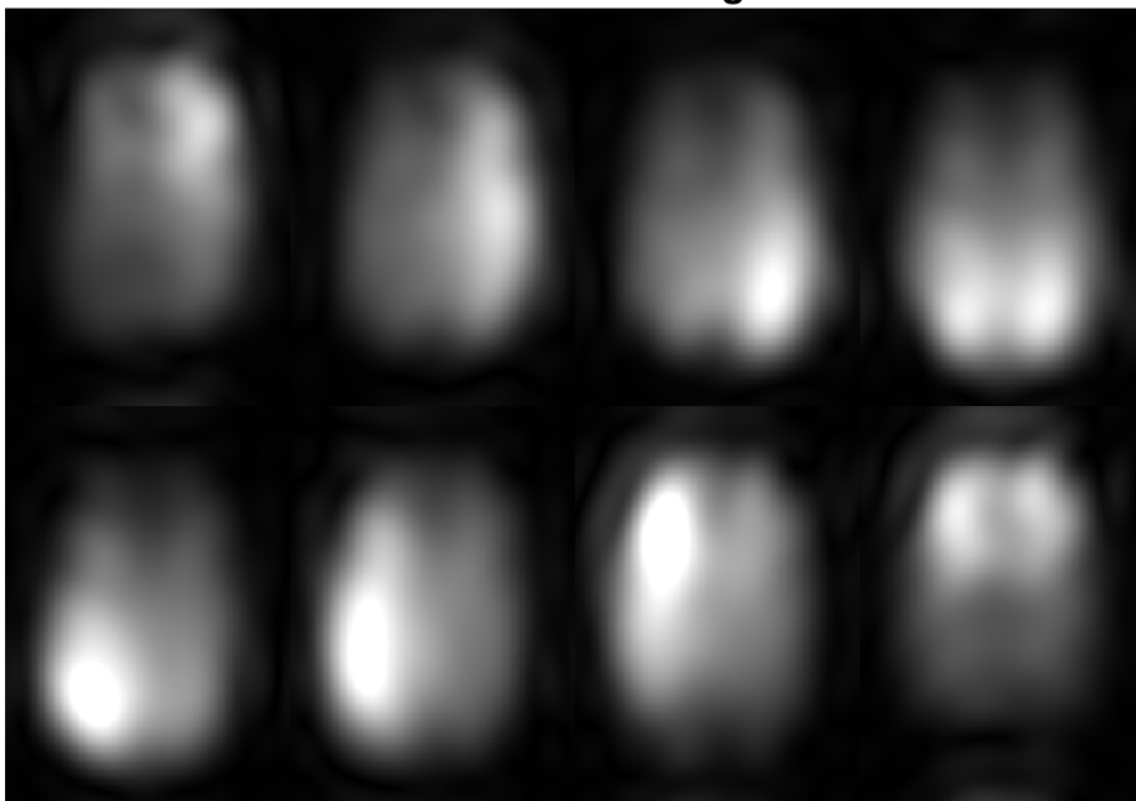


Figure 34: Calibration images using an 8×8 central k-space region.

Part 3.1: calibration k-space 8x8

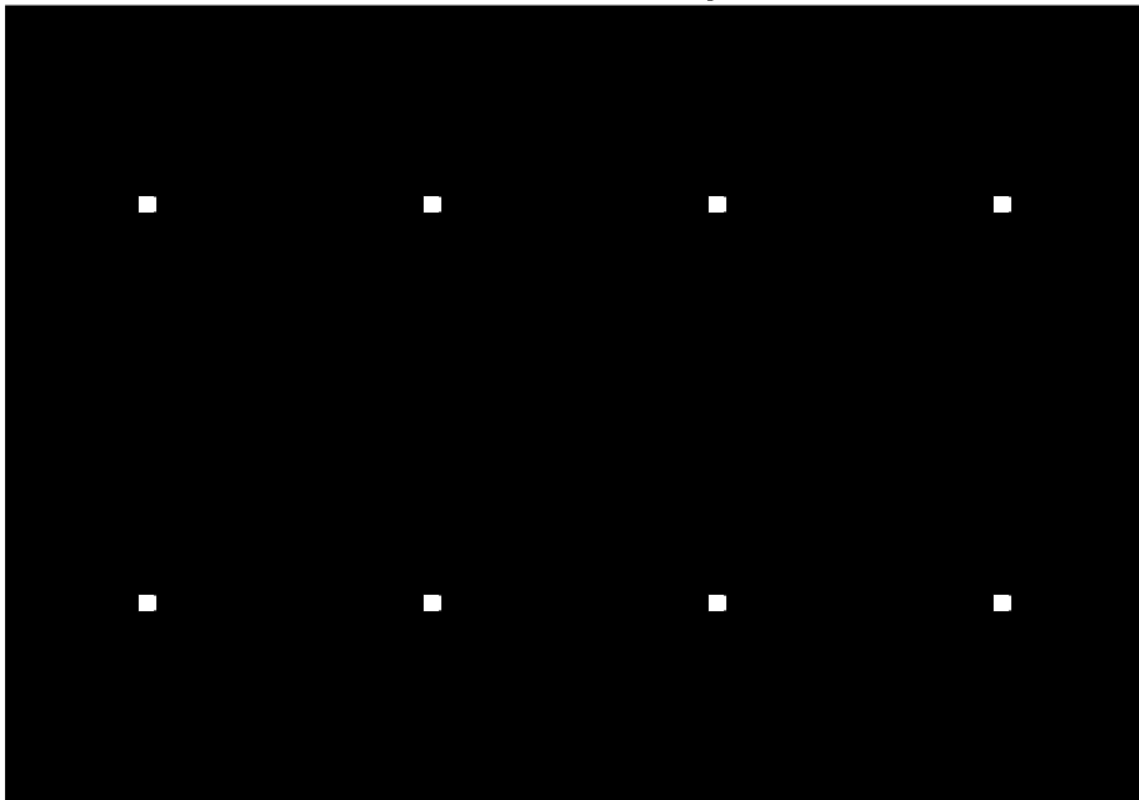


Figure 35: Calibration k-space spectra using an 8×8 central region.

Part 3.1: calibration images 16x16

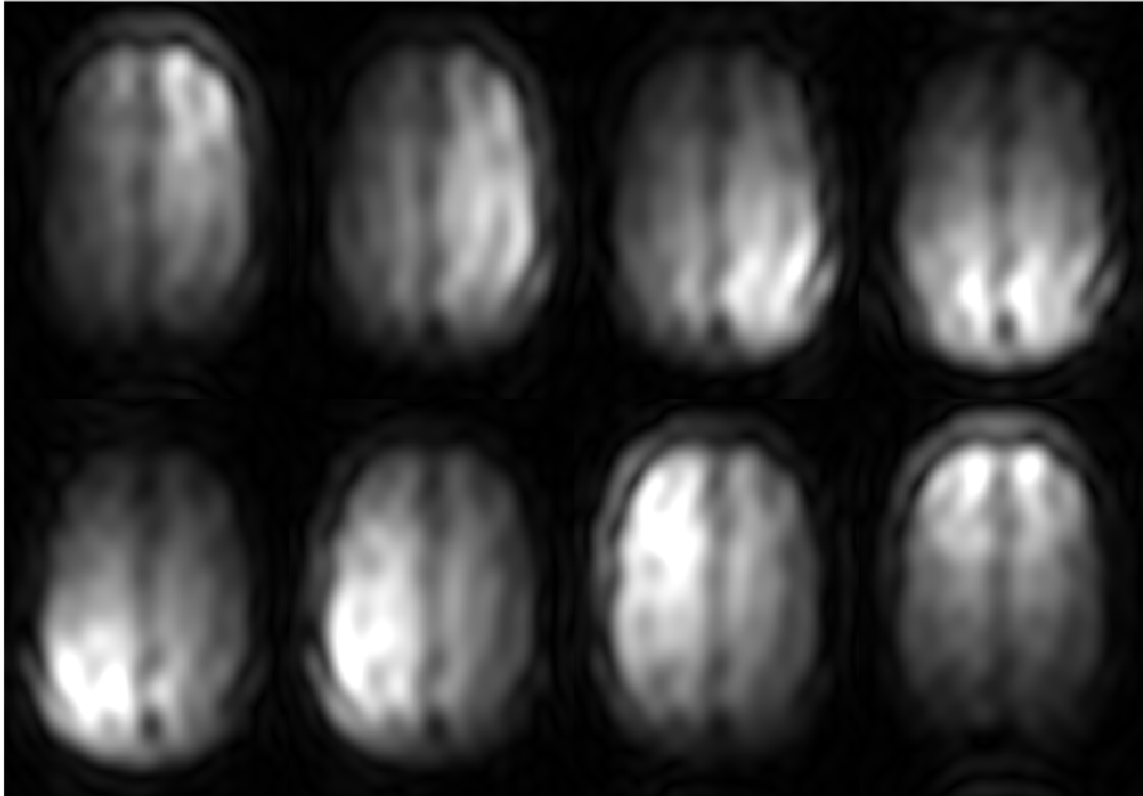


Figure 36: Calibration images using a 16×16 central k-space region.

Part 3.1: calibration k-space 16x16

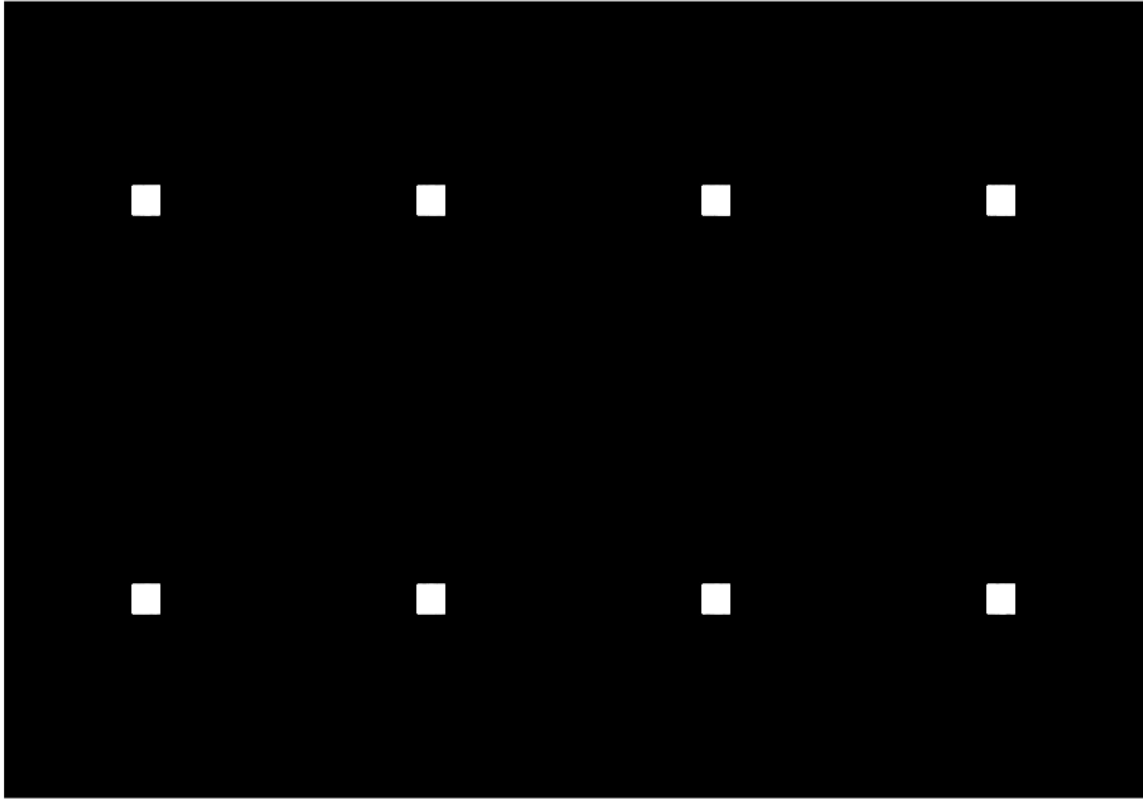


Figure 37: Calibration k-space spectra using a 16×16 central region.

Part 3.1: calibration images 32x32

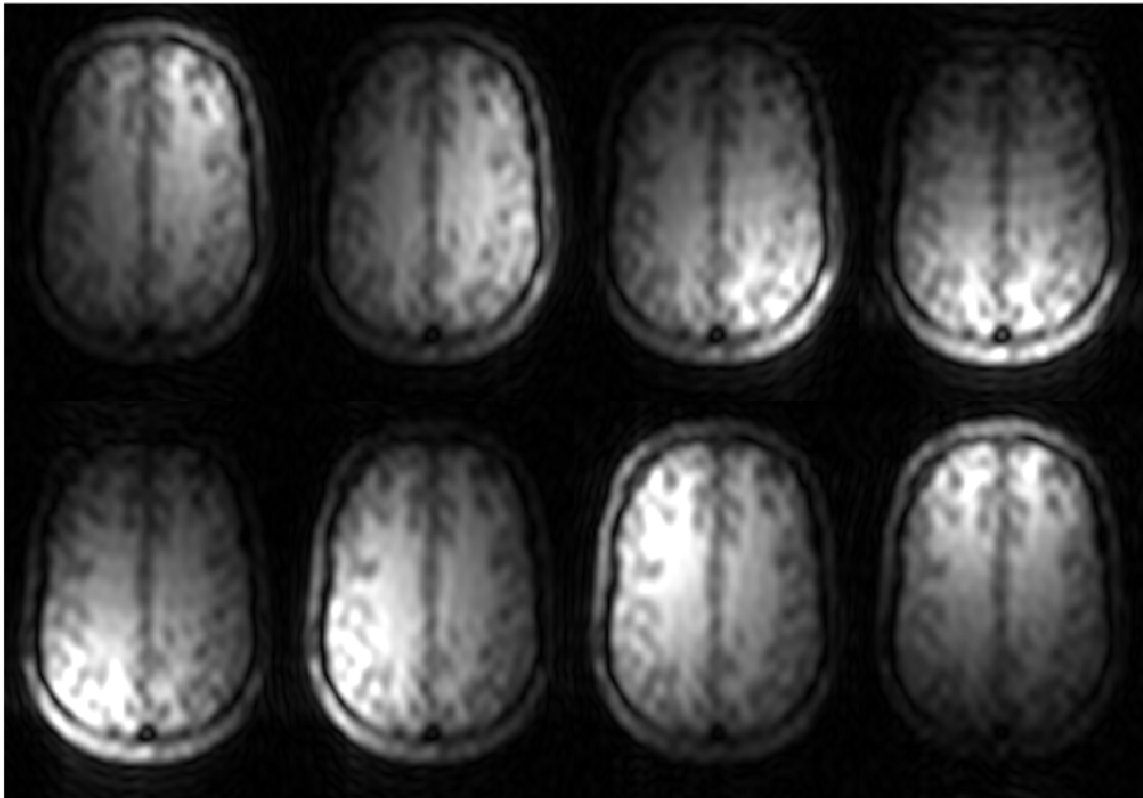


Figure 38: Calibration images using a 32×32 central k-space region.

Part 3.1: calibration k-space 32x32

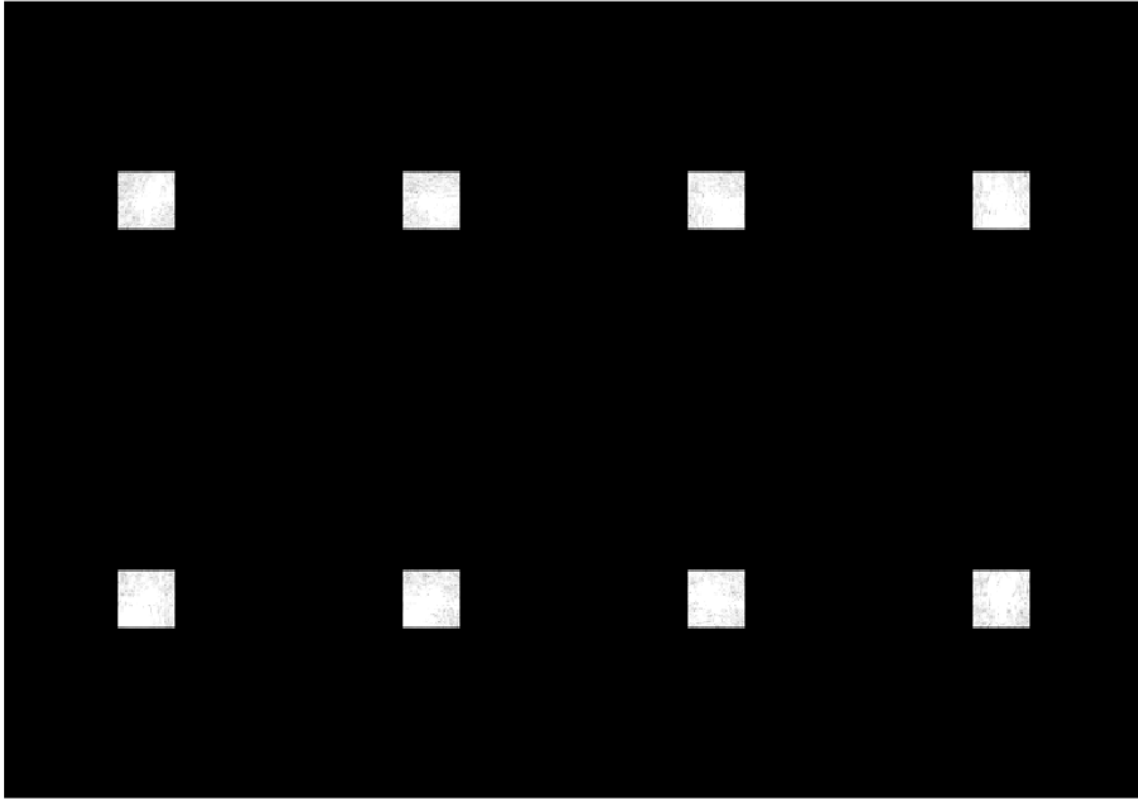


Figure 39: Calibration k-space spectra using a 32×32 central region.

Code

```
1 calib_cases = [8 8; 16 16; 32 32];
2 calib_data = struct();
3
4 for caseIdx = 1:size(calib_cases, 1)
5     calibx = calib_cases(caseIdx, 1);
6     caliby = calib_cases(caseIdx, 2);
7     [imc, Mc] = imcalib(im, calibx, caliby);
8
9     fieldName = sprintf('calib%d_%d', calibx, caliby);
10    calib_data.(fieldName).imc = imc;
11    calib_data.(fieldName).Mc = Mc;
12
13    show_montage(abs(imc), [], sprintf('Part_3.1: calibration_images_%dx%d', calibx,
14    caliby), ...
15    fullfile(figDir, sprintf('part3_1_imc_%dx%d.png', calibx, caliby)));
16    show_montage(kspace_spectrum(Mc), [], sprintf('Part_3.1: calibration_k-space_%dx%d',
17    calibx, caliby), ...
18    fullfile(figDir, sprintf('part3_1_Mc_%dx%d.png', calibx, caliby)));
19 end
```

3.2 Undersampled Images with Calibration Region

The central calibration region improves low-frequency consistency because the middle of k-space is fully sampled. The outer k-space still follows the accelerated sampling pattern, so aliasing and resolution loss remain visible. Increasing the acceleration rate removes more outer k-space samples and makes the aliasing stronger.

Part 3.2: undersampled+calib images $R_x=1$, $R_y=2$

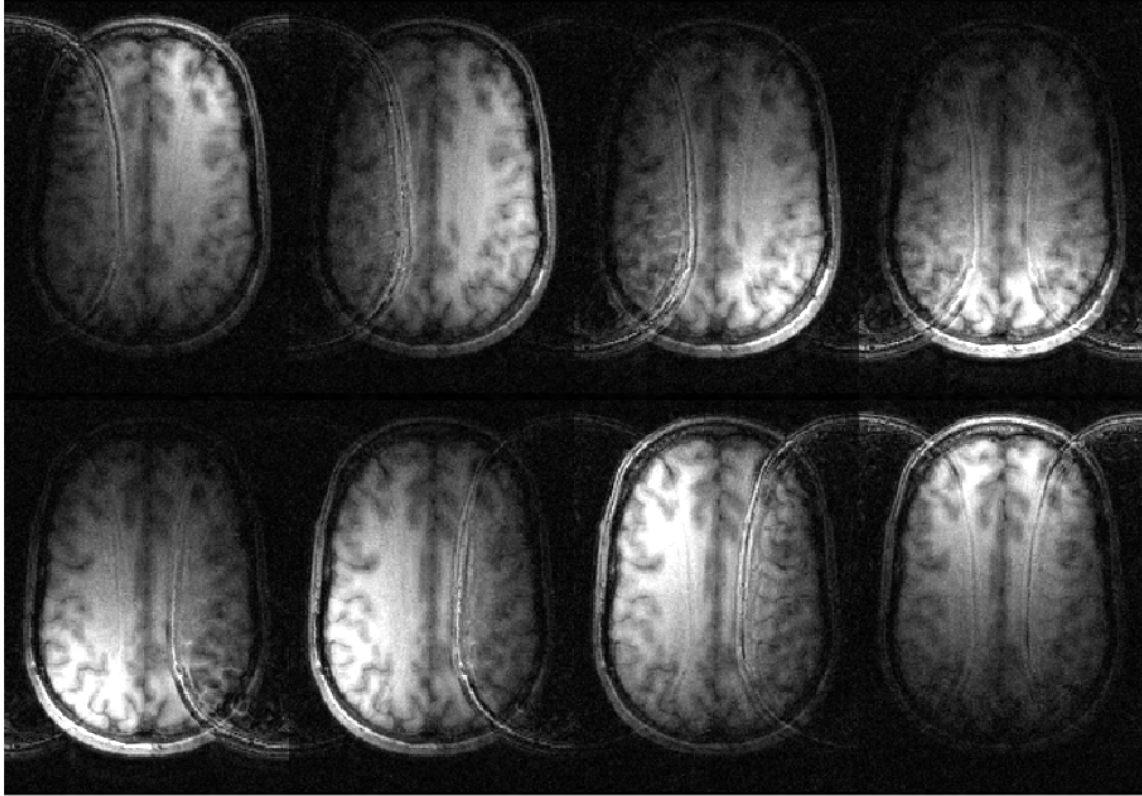


Figure 40: Undersampled images with 32×32 calibration region for $R_x = 1$, $R_y = 2$.

Part 3.2: undersampled+calib k-space Rx=1, Ry=2

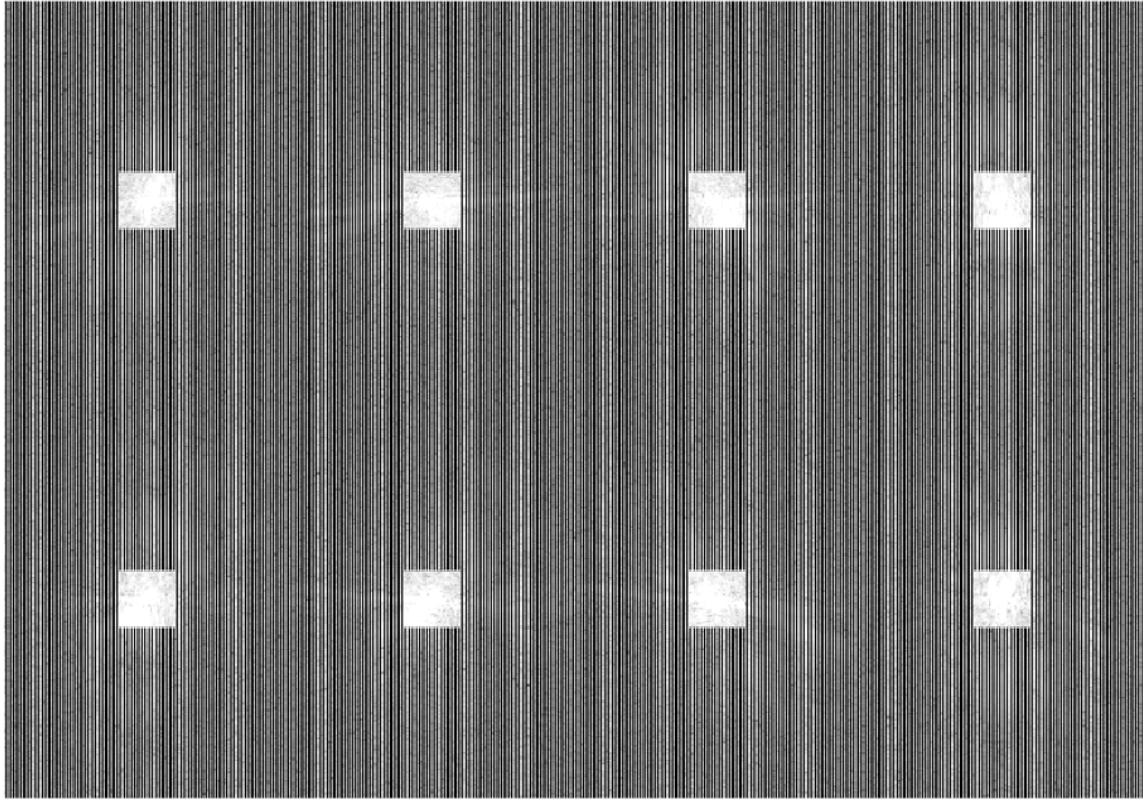


Figure 41: Undersampled k-space with 32×32 calibration region for $R_x = 1$, $R_y = 2$.

Part 3.2: undersampled+calib images $R_x=1$, $R_y=4$

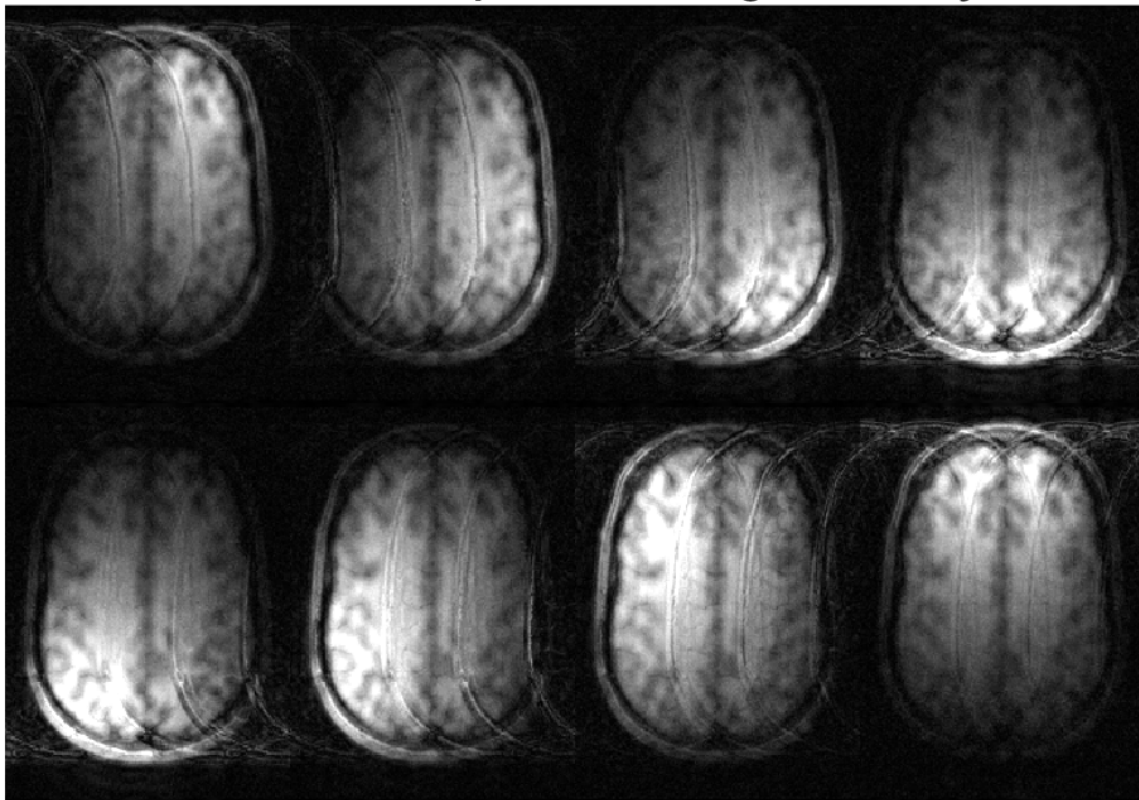


Figure 42: Undersampled images with 32×32 calibration region for $R_x = 1$, $R_y = 4$.

Part 3.2: undersampled+calib k-space Rx=1, Ry=4

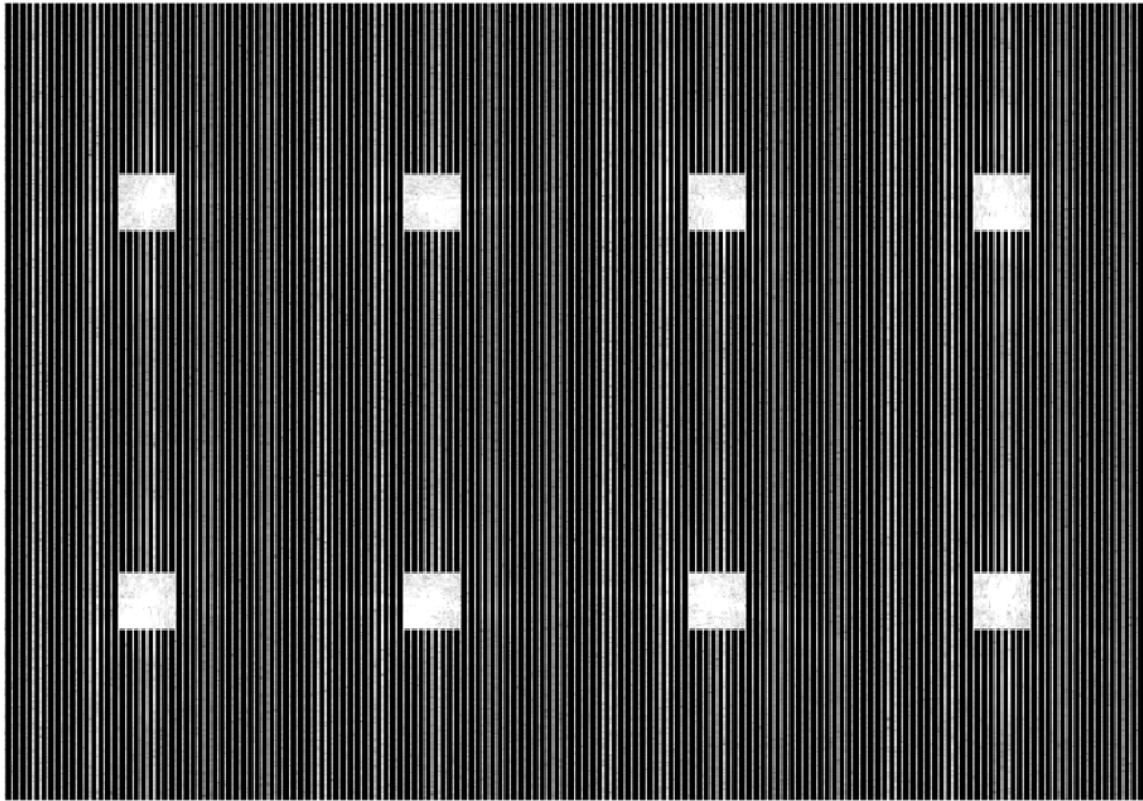


Figure 43: Undersampled k-space with 32×32 calibration region for $R_x = 1$, $R_y = 4$.

Part 3.2: undersampled+calib images $R_x=2, R_y=2$

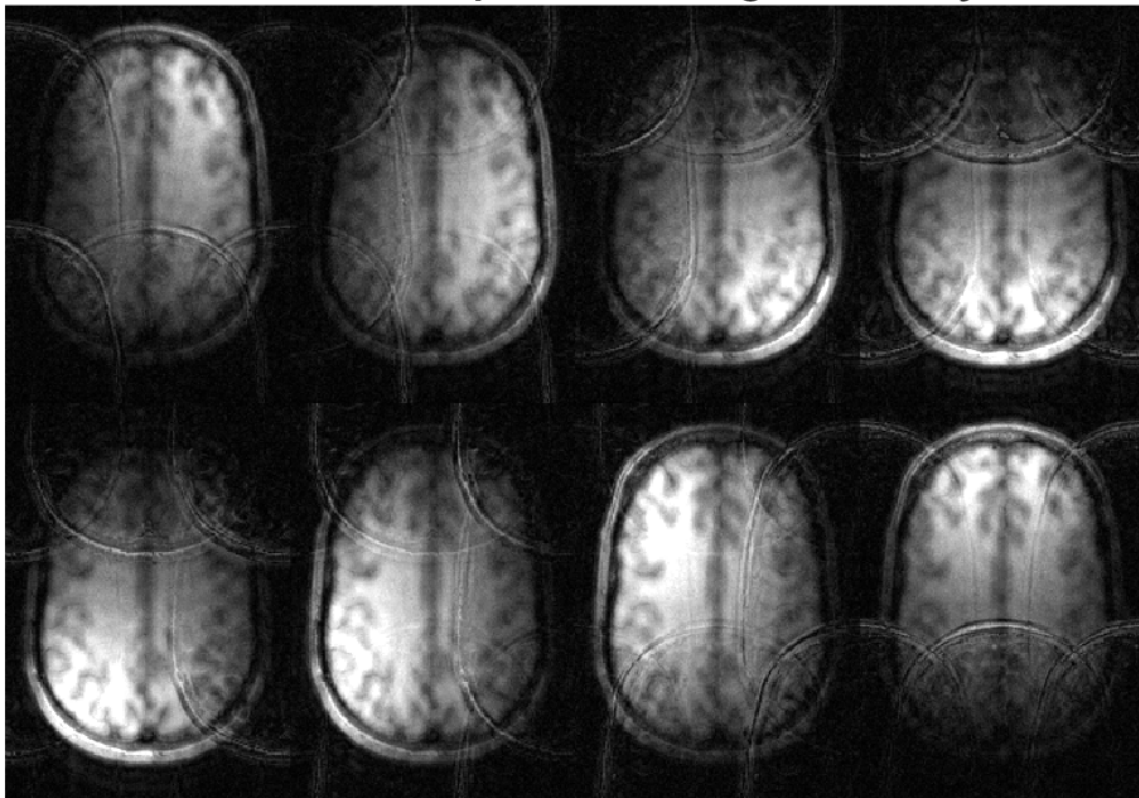


Figure 44: Undersampled images with 32×32 calibration region for $R_x = 2, R_y = 2$.

Part 3.2: undersampled+calib k-space $R_x=2, R_y=2$

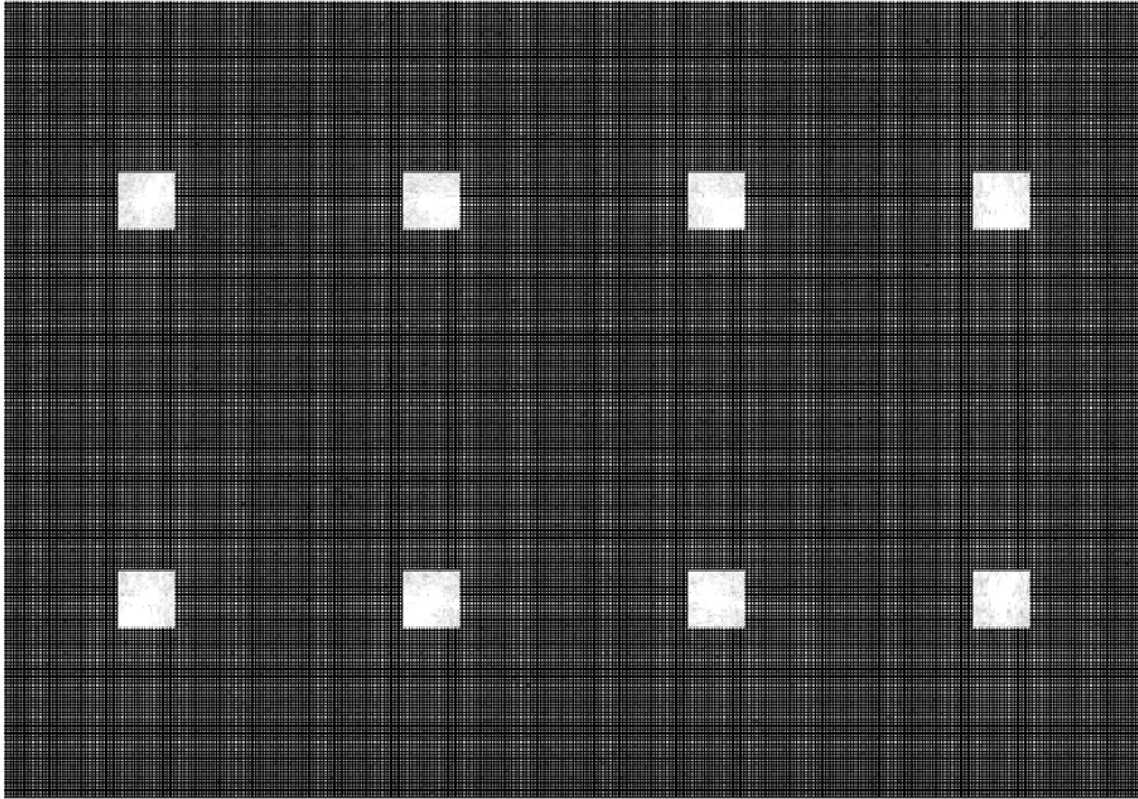


Figure 45: Undersampled k-space with 32×32 calibration region for $R_x = 2, R_y = 2$.

Code

```

1 calibx = 32;
2 caliby = 32;
3 undersampled_calib_tests = struct();
4
5 for caseIdx = 1:size(sense_cases, 1)
6     Rx = sense_cases(caseIdx, 1);
7     Ry = sense_cases(caseIdx, 2);
8     [imu, Mu] = undersamplecalib(im, Rx, Ry, calibx, caliby);
9
10    fieldName = sprintf('Rx%d_Ry%d', Rx, Ry);
11    undersampled_calib_tests.(fieldName).imu = imu;
12    undersampled_calib_tests.(fieldName).Mu = Mu;
13
14    show_montage(abs(imu), [], sprintf('Part_3.2:undersampled+calib_images_Rx=%d,Ry=%d',
15    Rx, Ry), ...
16    fullfile(figDir, sprintf('part3_2_images_Rx%d_Ry%d.png', Rx, Ry)));
17    show_montage(kspace_spectrum(Mu), [], sprintf('Part_3.2:undersampled+calib_k-space_
18    Rx=%d,Ry=%d', Rx, Ry), ...
19    fullfile(figDir, sprintf('part3_2_kspace_Rx%d_Ry%d.png', Rx, Ry)));
20 end

```

3.3 GRAPPA Kernel Weights

For $R_x = 1, R_y = 2$, every other k_y line is missing outside the ACS region. The GRAPPA kernel uses three neighboring readout positions from each of the two adjacent acquired k_y lines, giving six spatial source positions per coil. For $N_c = 8$, the source vector length is $6N_c = 48$. For $N_c = 8$, the GRAPPA kernel matrix has size 48×8 . The displayed kernel magnitude and phase show how source samples from different coils contribute to the reconstruction of each output coil.

MATLAB output: GRAPPA kernel size = 48×8 .

Part 3.3: GRAPPA kernel magnitude, lambda=0

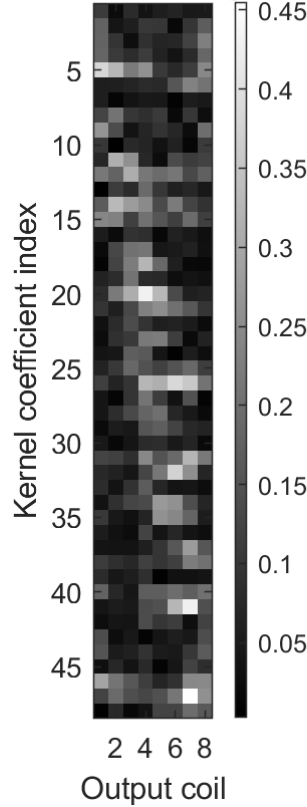


Figure 46: Magnitude of the GRAPPA kernel for $\lambda = 0$.

Part 3.3: GRAPPA kernel phase, lambda=0

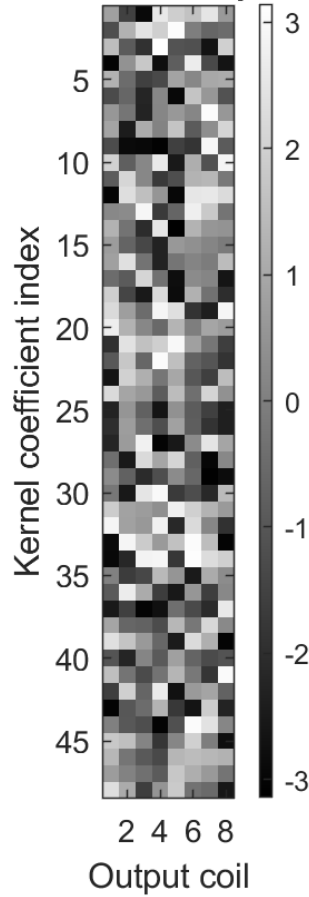


Figure 47: Phase of the GRAPPA kernel for $\lambda = 0$.

Code

```

1 [~, Mc32] = imcalib(im, 32, 32);
2 kernel0 = grappa_calibrate(Mc32, 0);
3 fprintf('Part_3.3:_GRAPPA_kernel_size=%d_x%d\n', size(kernel0, 1), size(kernel0, 2));
4
5 show_matrix_image(abs(kernel0), [], 'Part_3.3:_GRAPPA_kernel_magnitude,_lambda=0', ...
6     fullfile(figDir, 'part3_3_kernel_magnitude_lambda0.png'));
7 show_matrix_image(angle(kernel0), [-pi pi], 'Part_3.3:_GRAPPA_kernel_phase,_lambda=0',
8     ...
9     fullfile(figDir, 'part3_3_kernel_phase_lambda0.png'));

```

3.4 GRAPPA Reconstruction with $\lambda = 0$

After GRAPPA fills the missing k-space samples, each coil image is reconstructed by inverse centered FFT and combined using SoS. The reconstructed k-space spectra show that the missing outer k-space lines are filled using the calibrated kernel while originally acquired and ACS samples are preserved. GRAPPA substantially reduces the aliasing seen in simple undersampling, although residual errors can remain near edges and high-frequency structures because kernel interpolation is approximate.

Part 3.4: GRAPPA coil images, lambda=0

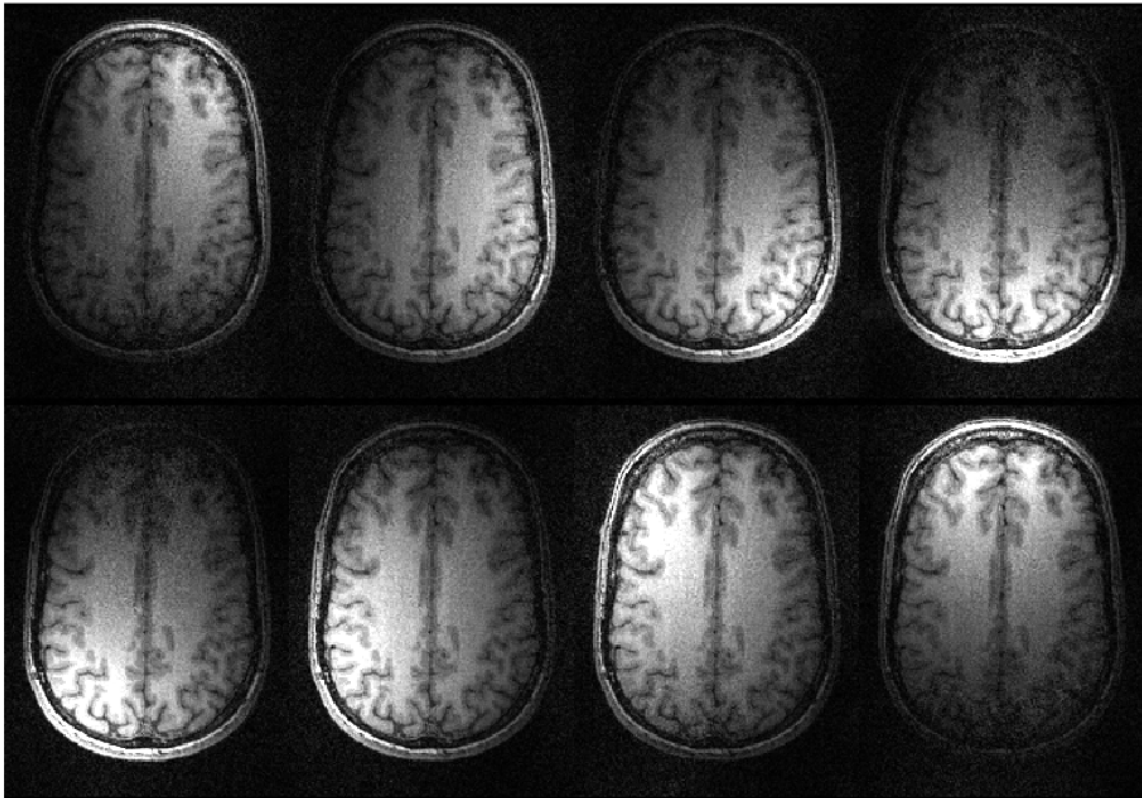


Figure 48: GRAPPA reconstructed coil images for $\lambda = 0$.

Part 3.4: GRAPPA reconstructed k-space, lambda=0

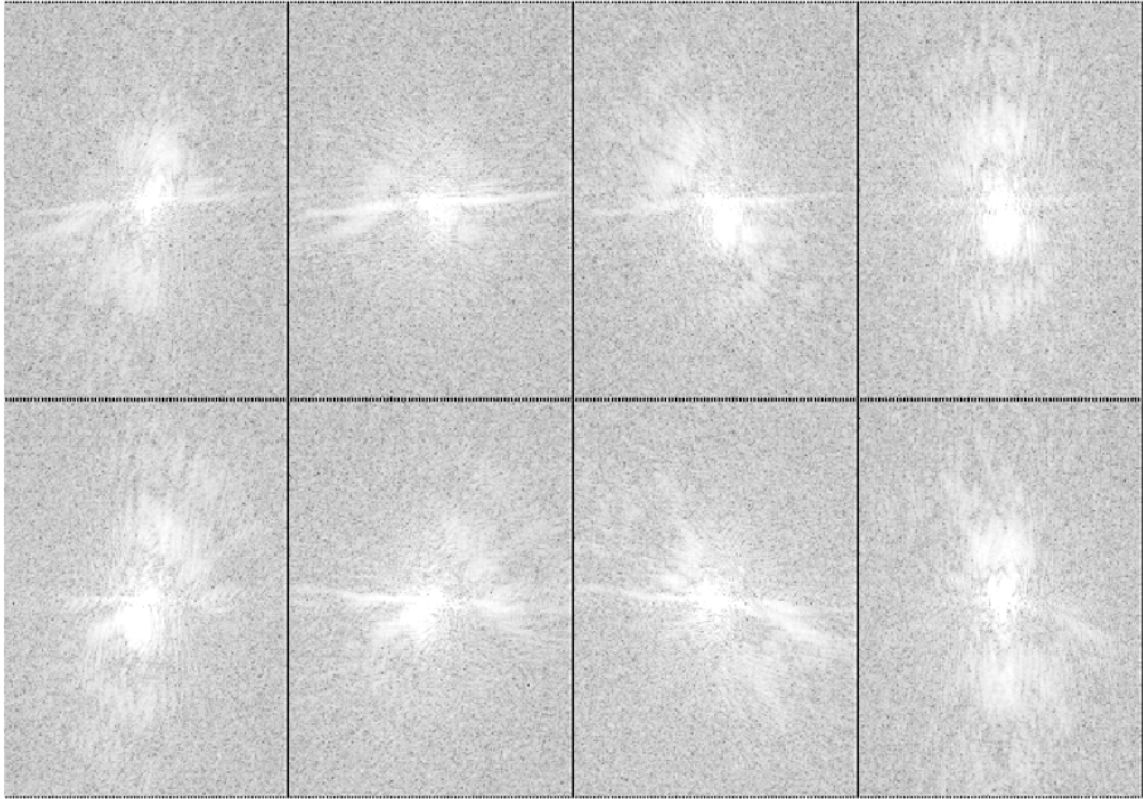


Figure 49: GRAPPA reconstructed k-space spectra for $\lambda = 0$.

Part 3.4: GRAPPA SoS, lambda=0

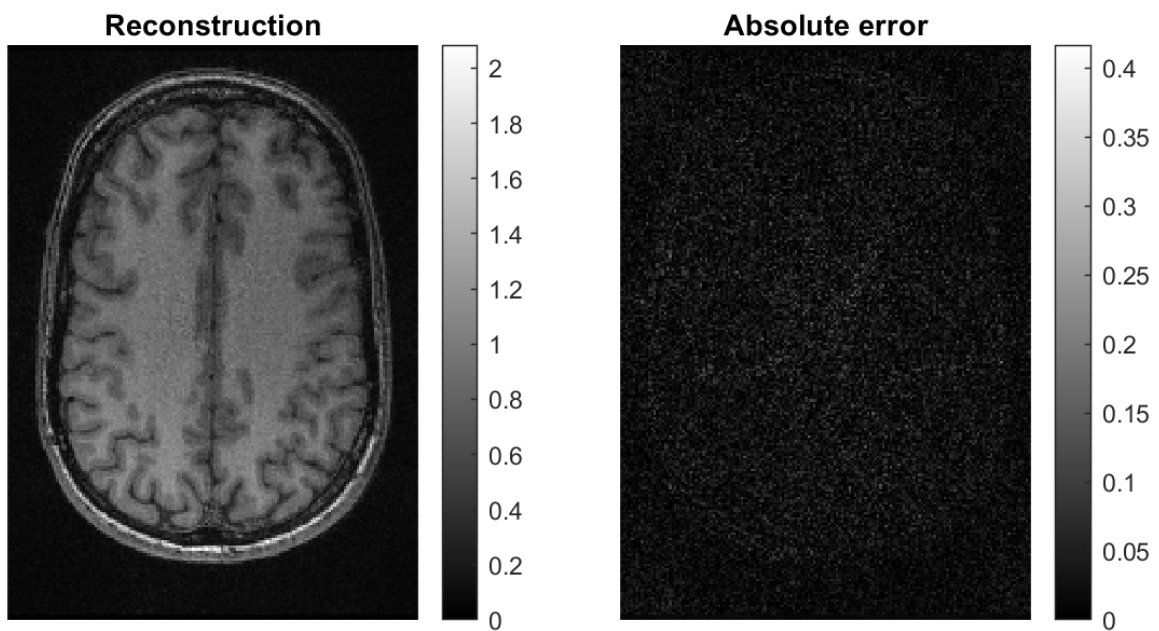


Figure 50: GRAPPA SoS reconstruction and error for $\lambda = 0$.

MATLAB output: $\lambda = 0$, PSNR = 35.4791 dB, SSIM = 0.916586.

Code

```
1 [~, Mc32] = imcalib(im, 32, 32);
2 [~, Mu_grappa] = undersamplecalib(im, 1, 2, 32, 32);
3 [imr0, Mr0] = grappa(Mu_grappa, Mc32, 0);
4
5 show_montage(abs(imr0), [], 'Part_3.4: GRAPPA_coil_images_lambda=0', ...
6     fullfile(figDir, 'part3_4_grappa_coil_images_lambda0.png'));
7 show_montage(kspace_spectrum(Mr0), [], 'Part_3.4: GRAPPA_reconstructed_k-space_lambda=0',
8     ...,
9     fullfile(figDir, 'part3_4_grappa_kspace_lambda0.png'));
10
11 grappa_sos0 = normalize95(sos_combine(imr0));
12 metrics_34 = compute_metrics(grappa_sos0, ref_grappa, max_val_grappa);
13
14 show_image_and_error(grappa_sos0, ref_grappa, max_val_grappa, ...
15     'Part_3.4: GRAPPA_SoS_lambda=0', ...
16     fullfile(figDir, 'part3_4_grappa_sos_lambda0.png'));
17
18 print_metrics('Part_3.4 GRAPPA_lambda=0', metrics_34);
```

3.5 GRAPPA Reconstruction with Regularization

Regularization is applied during kernel calibration by solving a ridge-regression problem for the GRAPPA weights. Initially, when I searched only a narrower range of λ values, nearly all tested values gave identical PSNR and SSIM, which made the regularization behavior suspicious. Therefore, I expanded the search to $\lambda \in \{0, 10^{-8}, \dots, 10^{15}\}$. The combined score was computed as the average of min-max normalized PSNR and min-max normalized SSIM values across the tested lambda values. Small and moderate λ values remain almost identical, but very large λ values eventually change the kernel enough to affect the reconstruction. The best combined score was obtained at $\lambda = 10^{11}$, which gives a small improvement over $\lambda = 0$. Since the improvement is modest, I do not interpret this as a major regularization benefit; it mainly shows that this data set is not very sensitive to regularization until extremely large λ values are used.

Part 3.5: GRAPPA reconstructed k-space, $\lambda=1e+11$

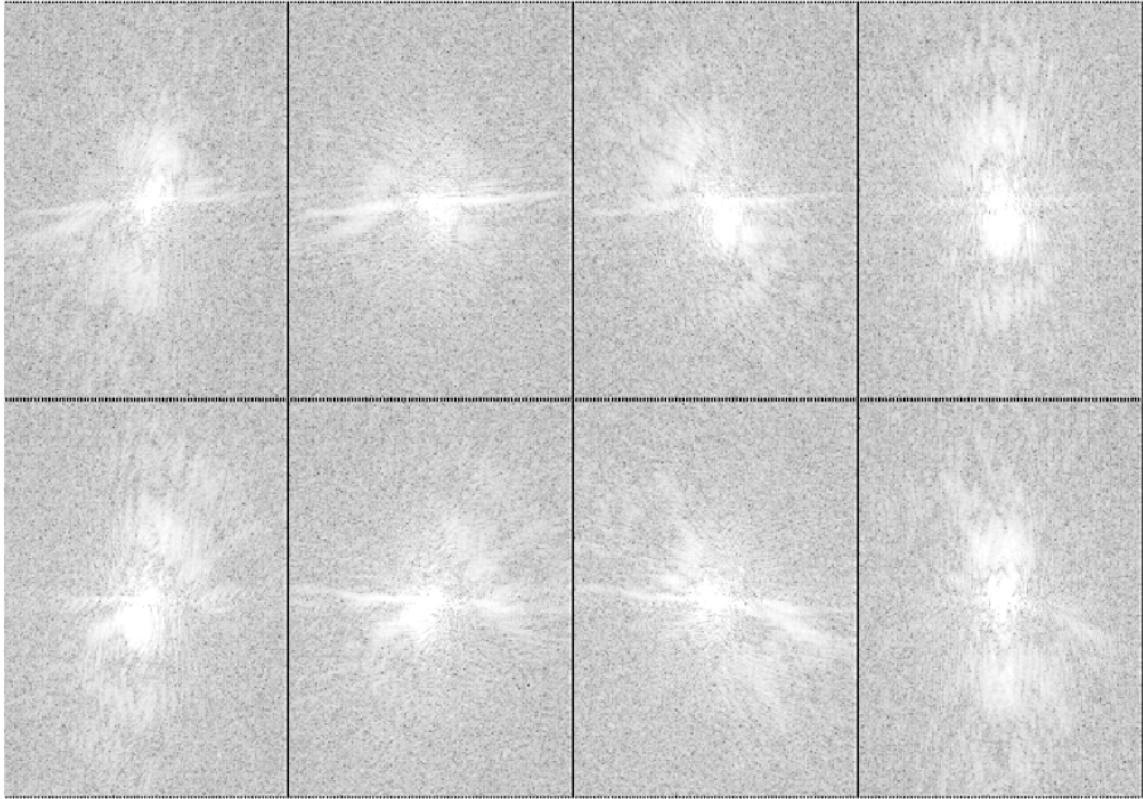


Figure 51: GRAPPA reconstructed k-space spectra using the selected regularization weight.

Part 3.5: GRAPPA SoS, $\lambda=1e+11$

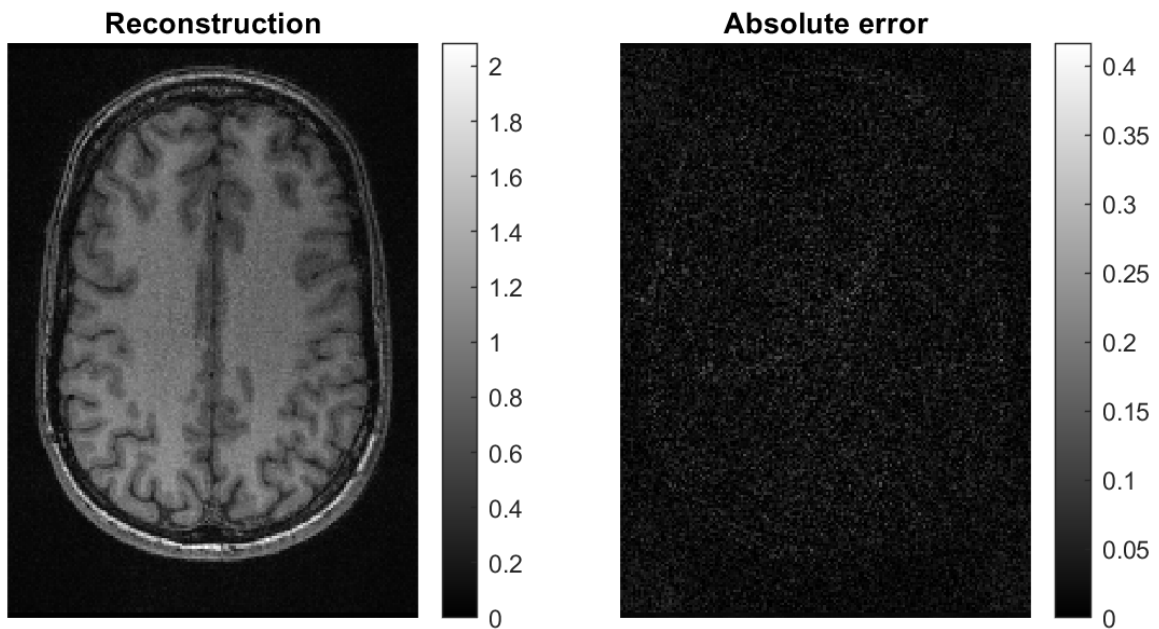


Figure 52: GRAPPA SoS reconstruction and error using the selected regularization weight.

MATLAB output: selected $\lambda = 10^{11}$, PSNR = 35.6017 dB, SSIM = 0.921049. The MATLAB code searches $\lambda \in \{0, 10^{-8}, \dots, 10^{15}\}$ and prints the full lambda table.

Code

```

1 lambda_candidates_grappa = [0, 10.^(-8:15)];
2 [~, Mc32] = imcalib(im, 32, 32);
3 [~, Mu_grappa] = undersamplecalib(im, 1, 2, 32, 32);
4
5 candidateImages = cell(numel(lambda_candidates_grappa), 1);
6 candidateKspace = cell(numel(lambda_candidates_grappa), 1);
7 grappa_scores = zeros(numel(lambda_candidates_grappa), 4);
8
9 for k = 1:numel(lambda_candidates_grappa)
10     lambda = lambda_candidates_grappa(k);
11     [imr, Mr] = grappa(Mu_grappa, Mc32, lambda);
12     grappa_sos = normalize95(sos_combine(imr));
13     metrics = compute_metrics(grappa_sos, ref_grappa, max_val_grappa);
14
15     candidateImages{k} = grappa_sos;
16     candidateKspace{k} = Mr;
17     grappa_scores(k, 1:3) = [lambda, metrics.psnr, metrics.ssim];
18 end
19
20 psnrVals = grappa_scores(:, 2);
21 ssimVals = grappa_scores(:, 3);
22 if max(psnrVals) - min(psnrVals) < 1e-3
23     psnrScore = zeros(size(psnrVals));
24 else
25     psnrScore = (psnrVals - min(psnrVals)) ./ (max(psnrVals) - min(psnrVals));
26 end
27 if max(ssimVals) - min(ssimVals) < 1e-4
28     ssimScore = zeros(size(ssimVals));
29 else
30     ssimScore = (ssimVals - min(ssimVals)) ./ (max(ssimVals) - min(ssimVals));
31 end
32 grappa_scores(:, 4) = 0.5 * psnrScore + 0.5 * ssimScore;
33
34 [~, bestIdx] = max(grappa_scores(:, 4));
35 bestLambda = grappa_scores(bestIdx, 1);
36 bestImage = candidateImages{bestIdx};
37 bestKspace = candidateKspace{bestIdx};
38 bestMetrics = struct('psnr', grappa_scores(bestIdx, 2), 'ssim', grappa_scores(bestIdx, 3)
39 );
40 lambda_grappa_selected = bestLambda;
41 grappa_score_table = array2table(grappa_scores, ...
42     'VariableNames', {'lambda', 'PSNR_dB', 'SSIM', 'combined_score'});
43
44 show_montage(kspace_spectrum(bestKspace), [], ...
45     sprintf('Part_3.5: GRAPPA_reconstructed_k-space, lambda=%g', bestLambda), ...
46     fullfile(figDir, 'part3_5_grappa_kspace_selected.png'));
47 show_image_and_error(bestImage, ref_grappa, max_val_grappa, ...
48     sprintf('Part_3.5: GRAPPA_SoS, lambda=%g', bestLambda), ...
49     fullfile(figDir, 'part3_5_grappa_sos_selected.png'));

```

```

50 fprintf('\nPart_3.5_lambda_search_table_for_GRAPPA:\n');
51 disp(grappa_score_table);
52 fprintf('Part_3.5_selected_lambda=%g\n', bestLambda);
53 print_metrics('Part_3.5_GRAPPA_selected_lambda', bestMetrics);

```

3.6 Final Remarks

At the matching acceleration factor $R_x = 1, R_y = 2$, both SENSE and GRAPPA produce high-quality reconstructions. With accurate `map1` sensitivities, regularized SENSE gives PSNR = 35.5545 dB and SSIM = 0.957431. Regularized GRAPPA gives PSNR = 35.6017 dB and SSIM = 0.921049. Thus, the two methods have very similar PSNR in this case, while SENSE gives the higher SSIM. GRAPPA also performs well and has the advantage that it is autocalibrated and does not require explicit coil sensitivity maps. The comparison shows that SENSE can perform very well when coil maps are accurate and the encoding is well-conditioned, while GRAPPA is useful when explicit sensitivity estimation is difficult or unreliable. Performance depends on acceleration rate, ACS size, coil geometry, regularization, and implementation details.

Shared MATLAB Helper Functions

The main helper functions used by the cells are listed below for completeness.

```

1 function y = fft2c(x)
2     y = fftshift(fftshift(fft2(ifftshift(ifftshift(x, 1), 2)), 1), 2);
3 end
4
5 function y = ifft2c(x)
6     y = fftshift(fftshift(ifft2(ifftshift(ifftshift(x, 1), 2)), 1), 2);
7 end
8
9 function s = kspace_spectrum(M)
10    s = log(1 + abs(M));
11 end
12
13 function out = normalize95(x)
14    out = abs(x);
15    out(~isfinite(out)) = 0;
16    p = prctile(out(:), 95);
17    if p > 0
18        out = out ./ p;
19    end
20    out(~isfinite(out)) = 0;
21 end
22
23 function out = olc_recon(im, map)
24    numerator = sum(conj(map) .* im, 3);
25    denominator = sum(abs(map).^2, 3);
26    out = numerator ./ denominator;
27    out(~isfinite(out)) = 0;
28 end
29
30 function out = sos_combine(im)
31    out = sqrt(sum(abs(im).^2, 3));

```

```

32     out(~isfinite(out)) = 0;
33 end
34
35 function [imu, Mu] = undersample(im, Rx, Ry)
36     [Nx, Ny, ~] = size(im);
37     M = fft2c(im);
38     xidx = centered_sampling_indices(Nx, Rx);
39     yidx = centered_sampling_indices(Ny, Ry);
40     Mu = M(xidx, yidx, :);
41     imu = ifft2c(Mu);
42     imu(~isfinite(imu)) = 0;
43 end
44
45 function [imc, Mc] = imcalib(im, calibx, caliby)
46     [Nx, Ny, Nc] = size(im);
47     M = fft2c(im);
48     mask = false(Nx, Ny);
49     xidx = center_indices(Nx, calibx);
50     yidx = center_indices(Ny, caliby);
51     mask(xidx, yidx) = true;
52     Mc = M .* repmat(mask, [1 1 Nc]);
53     imc = ifft2c(Mc);
54     imc(~isfinite(imc)) = 0;
55 end
56
57 function [imu, Mu] = undersamplecalib(im, Rx, Ry, calibx, caliby)
58     [Nx, Ny, Nc] = size(im);
59     M = fft2c(im);
60     mask = false(Nx, Ny);
61     xSample = centered_sampling_indices(Nx, Rx);
62     ySample = centered_sampling_indices(Ny, Ry);
63     mask(xSample, ySample) = true;
64     xidx = center_indices(Nx, calibx);
65     yidx = center_indices(Ny, caliby);
66     mask(xidx, yidx) = true;
67     Mu = M .* repmat(mask, [1 1 Nc]);
68     imu = ifft2c(Mu);
69     imu(~isfinite(imu)) = 0;
70 end
71
72 function im_sense = l2sense(imu, map, Rx, Ry, lambda)
73     [Nx, Ny, Nc] = size(map);
74     Nxu = size(imu, 1);
75     Nyu = size(imu, 2);
76     if Nxu ~= Nx / Rx || Nyu ~= Ny / Ry
77         error('The aliased image size is inconsistent with Rx and Ry. ');
78     end
79     im_sense_complex = zeros(Nx, Ny);
80
81     for ax = 1:Nxu
82         srcx = alias_source_indices(ax, Nx, Rx);
83         for ay = 1:Nyu
84             srcy = alias_source_indices(ay, Ny, Ry);
85             [XX, YY] = ndgrid(srcx, srcy);

```

```

86     lin = sub2ind([Nx Ny], XX(:), YY(:));
87     nsrc = numel(lin);
88     if nsrc ~= Rx * Ry
89         error('Unexpected_number_of_unfolded_source_voxels.');
```

```

90     end
91
92     C = zeros(Nc, nsrc);
93     for coil = 1:Nc
94         tmp = map(:, :, coil);
95         C(coil, :) = tmp(lin).';
96     end
97
98     y = squeeze(imu(ax, ay, :));
99     y(~isfinite(y)) = 0;
100    if norm(C, 'fro') == 0 || ~any(y)
101        im_sense_complex(lin) = 0;
102        continue;
103    end
104
105    G = C' * C;
106    rhs = C' * y;
107    if lambda == 0
108        xhat = pinv(G) * rhs;
109    else
110        xhat = (G + lambda * eye(nsrc)) \ rhs;
111    end
112    im_sense_complex(lin) = xhat;
113 end
114 end
115
116 im_sense = normalize95(abs(im_sense_complex));
117 end
118
119 function g = gfactor(map, Rx, Ry, lambda)
120     [Nx, Ny, Nc] = size(map);
121     Nxu = Nx / Rx;
122     Nyu = Ny / Ry;
123     g = zeros(Nx, Ny);
124
125     for ax = 1:Nxu
126         srcx = alias_source_indices(ax, Nx, Rx);
127         for ay = 1:Nyu
128             srcy = alias_source_indices(ay, Ny, Ry);
129             [XX, YY] = ndgrid(srcx, srcy);
130             lin = sub2ind([Nx Ny], XX(:), YY(:));
131             nsrc = numel(lin);
132             if nsrc ~= Rx * Ry
133                 error('Unexpected_number_of_unfolded_source_voxels.');
```

```

134             end
135
136             C = zeros(Nc, nsrc);
137             for coil = 1:Nc
138                 tmp = map(:, :, coil);
139                 C(coil, :) = tmp(lin).';

```

```

140         end
141
142         G = C' * C;
143         B = pinv(G + lambda * eye(nsrc));
144         covSense = B * G * B';
145
146         for s = 1:nsrc
147             sensPower = real(G(s, s));
148             if sensPower <= 0
149                 g(lin(s)) = 0;
150             else
151                 gArg = real(sensPower * covSense(s, s));
152                 if gArg < 0 && abs(gArg) < 1e-10
153                     gArg = 0;
154                 end
155                 gval = sqrt(max(gArg, 0));
156                 if isfinite(gval)
157                     g(lin(s)) = gval;
158                 end
159             end
160         end
161     end
162 end
163
164 allZero = sum(abs(map).^2, 3) == 0;
165 g(allZero) = 0;
166 end
167
168 function stats = gfactor_stats(g, map)
169     objectMask = sum(abs(map).^2, 3) > 0;
170     vals = g(objectMask);
171     vals = vals(isfinite(vals));
172     if isempty(vals)
173         stats.max = 0;
174         stats.mean = 0;
175     else
176         stats.max = max(vals(:));
177         stats.mean = mean(vals(:));
178     end
179 end
180
181 function kernel = grappa_calibrate(Mc, lambda)
182     [Nx, Ny, Nc] = size(Mc);
183     mask = any(abs(Mc) > 0, 3);
184     rows = find(any(mask, 2));
185     cols = find(any(mask, 1));
186
187     if isempty(rows) || isempty(cols)
188         error('Calibration data are empty. ');
189     end
190
191     xRange = rows(1):rows(end);
192     yRange = cols(1):cols(end);
193     xTargets = xRange(2:end-1);

```

```

194 yTargets = yRange(2:end-1);
195 acquiredY = centered_sampling_indices(Ny, 2);
196 yTargets = yTargets(~ismember(yTargets, acquiredY));
197
198 nExamples = numel(xTargets) * numel(yTargets);
199 source = zeros(nExamples, 6 * Nc);
200 target = zeros(nExamples, Nc);
201
202 row = 0;
203 for x = xTargets
204     for y = yTargets
205         row = row + 1;
206         vec = zeros(1, 6 * Nc);
207         for coil = 1:Nc
208             base = (coil - 1) * 6;
209             vec(base + (1:6)) = grappa_source_samples(Mc, x, y, coil);
210         end
211         source(row, :) = vec;
212         target(row, :) = reshape(Mc(x, y, :), 1, Nc);
213     end
214 end
215
216 kernel = (source' * source + lambda * eye(6 * Nc)) \ (source' * target);
217 end
218
219 function [imr, Mr] = grappa(Mu, Mc, lambda)
220     [Nx, Ny, Nc] = size(Mu);
221     kernel = grappa_calibrate(Mc, lambda);
222     Mr = Mu;
223
224     calibMask = any(abs(Mc) > 0, 3);
225     rows = find(any(calibMask, 2));
226     cols = find(any(calibMask, 1));
227
228     acquiredMask = false(Nx, Ny);
229     acquiredMask(:, centered_sampling_indices(Ny, 2)) = true;
230     acquiredMask(rows(1):rows(end), cols(1):cols(end)) = true;
231
232     for x = 2:Nx-1
233         for y = 2:Ny-1
234             if ~acquiredMask(x, y)
235                 vec = zeros(1, 6 * Nc);
236                 for coil = 1:Nc
237                     base = (coil - 1) * 6;
238                     vec(base + (1:6)) = grappa_source_samples(Mr, x, y, coil);
239                 end
240                 Mr(x, y, :) = reshape(vec * kernel, 1, 1, Nc);
241             end
242         end
243     end
244
245     imr = ifft2c(Mr);
246     imr(~isfinite(imr)) = 0;
247 end

```

```

248
249 function samples = grappa_source_samples(M, x, y, coil)
250     samples = [ ...
251         M(x-1, y-1, coil), M(x, y-1, coil), M(x+1, y-1, coil), ...
252         M(x-1, y+1, coil), M(x, y+1, coil), M(x+1, y+1, coil)];
253 end
254
255 function idx = center_indices(N, width)
256     first = floor(N / 2) - floor(width / 2) + 1;
257     idx = first:(first + width - 1);
258 end
259
260 function idx = centered_sampling_indices(N, R)
261     if mod(N, R) ~= 0
262         error('Image_size_must_be_divisible_by_the_acceleration_factor.');
```

```

263     end
264     center = floor(N / 2) + 1;
265     idx = find(mod((1:N) - center, R) == 0);
266     if numel(idx) ~= N / R
267         error('Unexpected_number_of_retained_k-space_samples.');
```

```

268     end
269 end
270
271 function idx = alias_source_indices(aliasIndex, N, R)
272     M = N / R;
273     fullCenter = floor(N / 2) + 1;
274     aliasCenter = floor(M / 2) + 1;
275     first = mod(aliasIndex - aliasCenter + fullCenter - 1, M) + 1;
276     idx = first:M:N;
277 end
278
279 function metrics = compute_metrics(img, ref, maxVal)
280     a = real(img) ./ maxVal;
281     b = real(ref) ./ maxVal;
282     a(~isfinite(a)) = 0;
283     b(~isfinite(b)) = 0;
284
285     err = a - b;
286     mse = mean(err(:).^2);
287     if mse == 0
288         metrics.psnr = Inf;
289     else
290         metrics.psnr = 10 * log10(1 / mse);
291     end
292
293     try
294         metrics.ssim = ssim(a, b, 'DynamicRange', 1);
295     catch
296         try
297             metrics.ssim = ssim(a, b);
298         catch
299             metrics.ssim = NaN;
300             warning('SSIM_could_not_be_computed._Check_that_the_Image_Processing_Toolbox_
is_available.');
```

```

301         end
302     end
303 end
304
305 function print_metrics(label, metrics)
306     fprintf('%s: PSNR=%%.4f dB, SSIM=%%.6f\n', label, metrics.psnr, metrics.ssim);
307 end
308
309 function show_montage(stack, displayRange, figTitle, outPath)
310     figure('Color', 'w');
311     if isempty(displayRange)
312         hi = prctile(abs(stack(:)), 99);
313         if hi <= 0 || ~isfinite(hi)
314             hi = 1;
315         end
316         displayRange = [0 hi];
317     end
318     montage(stack, 'DisplayRange', displayRange);
319     colormap gray;
320     title(figTitle, 'Interpreter', 'none');
321     save_current_figure(outPath);
322 end
323
324 function show_single_image(img, displayRange, figTitle, outPath)
325     figure('Color', 'w');
326     imshow(img, displayRange);
327     colormap gray;
328     colorbar;
329     title(figTitle, 'Interpreter', 'none');
330     save_current_figure(outPath);
331 end
332
333 function show_image_and_error(img, ref, maxVal, figTitle, outPath)
334     figure('Color', 'w');
335     tiledlayout(1, 2, 'Padding', 'compact', 'TileSpacing', 'compact');
336
337     nexttile;
338     imshow(img, [0 maxVal]);
339     colormap gray;
340     title('Reconstruction', 'Interpreter', 'none');
341     colorbar;
342
343     nexttile;
344     imshow(abs(img - ref), [0 0.2 * maxVal]);
345     colormap gray;
346     title('Absolute_error', 'Interpreter', 'none');
347     colorbar;
348
349     sgtitle(figTitle, 'Interpreter', 'none');
350     save_current_figure(outPath);
351 end
352
353 function show_matrix_image(A, displayRange, figTitle, outPath)
354     figure('Color', 'w');

```

```

355     if isempty(displayRange)
356         imagesc(A);
357     else
358         imagesc(A, displayRange);
359     end
360     axis image;
361     colormap gray;
362     colorbar;
363     title(figTitle, 'Interpreter', 'none');
364     xlabel('Output_coil');
365     ylabel('Kernel_coefficient_index');
366     save_current_figure(outPath);
367 end
368
369 function save_current_figure(outPath)
370     [outDir, ~, ~] = fileparts(outPath);
371     ensure_dir(outDir);
372     try
373         exportgraphics(gcf, outPath, 'Resolution', 200);
374     catch
375         saveas(gcf, outPath);
376     end
377 end
378
379 function ensure_dir(pathName)
380     if ~exist(pathName, 'dir')
381         mkdir(pathName);
382     end
383 end

```